SOFTWARE REQUIREMENTS SPECIFICATION

Design and Implementation of an Edge-Based Cyber-Physical Digital Twin Framework for Autonomous Warehouse AGVs using Raspberry Pi 5 and ESP32

Document ID	DOC-SRS-AGV-DT-2026-001
Document Type	Software Requirements Specification (SRS)
Project Type	Final Year / Research Project — Embedded Systems & Robotics
Department	[Department of Electronics / Computer Science — to be filled]
Institution	[University / Institution Name — to be filled]
Team Members	[Student Name 1], [Student Name 2], [Student Name 3]
Project Guide	[Guide / Professor Name — to be filled]
Academic Year	2025–2026
Version	1.0 (Draft for Approval)
Status	Under Review
Date	August 2026
Classification	Academic / Confidential

This document has been prepared for academic submission and project development. All hardware specifications, performance targets, and design decisions described herein are proposed values unless explicitly stated otherwise. No test results, research outcomes, or performance measurements have been fabricated. This specification is intended to guide the development team through all phases of implementation.

Document Control

Revision History

Version	Date	Author	Reviewer	Status	Description of Change
1.0	Aug 2026	[Author(s)]	[Reviewer]	Draft	Initial SRS draft for academic submission
1.1	TBD	[Author(s)]	[Reviewer]	Pending	Post-review corrections
2.0	TBD	[Author(s)]	[Reviewer]	Pending	Final approved version

Referenced Documents

Ref ID	Document Title	Version	Remarks
REF-01	IEEE Std 830-1998 — Recommended Practice for SRS	1998	SRS structure guideline
REF-02	ESP32 Technical Reference Manual	v5.x	Microcontroller firmware reference
REF-03	Raspberry Pi 5 Hardware Documentation	Latest	Edge node hardware reference
REF-04	MQTT Protocol Specification OASIS v3.1.1	3.1.1	Communication protocol
REF-05	ISO 3691-4:2020 Industrial Truck Safety	2020	Safety reference (informative only)

Approval Signatures

Role	Name	Signature	Date
Project Guide / Supervisor	[Professor Name]	_____	_____
Team Lead	[Student Name]	_____	_____
HOD / Evaluator	[HoD Name]	_____	_____

Table of Contents

(Auto-generated in final build)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete set of functional, non-functional, hardware, embedded, communication, and safety requirements for the **CyberTwin-AGV** project, formally titled: *Design and Implementation of an Edge-Based Cyber-Physical Digital Twin Framework for Autonomous Warehouse AGVs using Raspberry Pi 5 and ESP32*. This document serves dual purposes: (a) as a submission-ready specification for academic project approval, and (b) as the primary technical reference for all development phases from hardware procurement through final validation and research experiment execution.

The intended audiences for this SRS are the project development team, academic supervisors, project evaluators, and any external collaborators involved in the design, implementation, or assessment of the system.

1.2 Project Background

1.2.1 Autonomous Guided Vehicles (AGVs)

Autonomous Guided Vehicles are mobile robotic platforms designed to transport goods within structured environments such as warehouses, manufacturing floors, and logistics centres without continuous human intervention. Modern AGVs integrate a range of sensing, actuation, and decision-making capabilities to navigate defined routes, avoid obstacles, and execute material-handling missions. The adoption of AGVs has accelerated significantly with the growth of e-commerce and the increasing demand for efficient, scalable warehouse automation.

1.2.2 Smart Warehouses and Industry 4.0

Industry 4.0 represents the fourth industrial revolution, characterised by the convergence of cyber-physical systems (CPS), the Internet of Things (IoT), cloud computing, and advanced analytics in industrial settings. Smart warehouses embody these principles by integrating automated material-handling systems, real-time inventory management, and data-driven operational intelligence. Within this paradigm, AGVs form a critical component of the physical automation layer, while digital representation and monitoring systems provide visibility and control at the supervisory layer.

1.2.3 Cyber-Physical Systems (CPS)

A Cyber-Physical System is an engineered system that tightly integrates physical processes with computational elements and networked communication. In a CPS, sensors observe the state of the physical world, computational nodes process and reason over this data, and actuators effect physical change based on decisions made in the digital domain. The CyberTwin-AGV system is explicitly designed as a CPS in which the physical AGV and its digital twin operate in continuous bidirectional synchronisation.

1.2.4 Edge Computing

Edge computing refers to the paradigm of performing computation at or near the source of data generation, rather than transmitting all raw data to a centralised cloud server. In the context of this project, a Raspberry Pi 5 single-board computer serves as the on-premises edge node, performing local data

aggregation, telemetry processing, navigation support, MQTT brokerage, database operations, and digital twin synchronisation. This design minimises communication latency and reduces dependence on external cloud infrastructure, making the system operable in network-constrained environments.

1.2.5 Digital Twins

A Digital Twin (DT) is a dynamic, data-driven virtual representation of a physical entity or system that mirrors the real-world state in near real-time and provides a platform for monitoring, analysis, simulation, and control. Unlike a simple dashboard that displays sensor values, a Digital Twin maintains a semantically rich model of the physical entity — including its state, history, mission context, and health — and supports bidirectional interaction: the physical system updates the twin's state, and commands from the twin's management interface propagate back to the physical system. The synchronisation fidelity and latency between the physical entity and its digital counterpart are central research metrics in this project.

1.3 Problem Statement

Conventional miniature AGV and robotic warehouse demonstrators — as implemented in academic and research contexts — typically suffer from several systemic limitations that restrict their value as research platforms and reduce their fidelity as representations of industrial reality:

- **Limited Real-Time Visibility:** Sensor data is often displayed locally on a serial monitor or basic display without structured telemetry pipelines, preventing remote monitoring and analysis.
- **Lack of Synchronised Digital Representation:** No persistent, live digital model of the AGV and its operating environment is maintained, preventing the study of synchronisation behaviour, state drift, or twin fidelity.
- **Centralised or Cloud-Dependent Processing:** Many academic demonstrators rely either on a host PC connected via USB or on cloud services, both of which introduce unnecessary latency and single points of failure.
- **Absence of Mission Lifecycle Management:** Warehouse operations involve structured missions with defined states, priorities, and outcomes. Simple robot systems lack this abstraction layer entirely.
- **Inadequate Historical Data Management:** Without time-series storage and structured event logging, retrospective analysis of navigation performance, fault events, and battery characteristics is impractical.
- **No Bidirectional Command Channel:** Most demonstrators are purely telemetry sinks — they report state but provide no structured interface for issuing commands, pausing missions, or updating navigation goals from a supervisory interface.
- **Lack of Measurable Research Metrics:** Without a defined experimental framework, parameters such as synchronisation latency, localisation accuracy, and communication reliability cannot be systematically measured or reported.

1.4 Proposed Solution

The CyberTwin-AGV framework addresses these limitations through a hierarchical, layered architecture that integrates embedded real-time control, edge computing, structured MQTT-based telemetry, a semantically rich digital twin, and a web-based supervisory dashboard. The architecture is organised into five functional layers:

Layer	Component	Primary Role

L1 — Physical	AGV chassis, motors, sensors	Physical actuation and sensing
L2 — Embedded	ESP32 microcontroller	Real-time motor control, sensor acquisition, safety
L3 — Edge	Raspberry Pi 5	Data aggregation, MQTT, navigation, DB, DT sync
L4 — Digital Twin	Virtual AGV + Warehouse model	State mirroring, history, analytics
L5 — Application	Web dashboard (React/HTML)	Visualisation, mission control, diagnostics

The ESP32 communicates with the Raspberry Pi 5 over UART. The Raspberry Pi hosts an MQTT broker (Mosquitto), a Python-based telemetry processing service, a FastAPI REST backend, a time-series database (InfluxDB) and/or a relational database (SQLite/PostgreSQL), and a digital twin synchronisation engine. The dashboard communicates with the backend over HTTP/WebSocket and enables operators to monitor the AGV, create and manage missions, and issue commands that propagate back to the physical AGV via the MQTT→ESP32→motor control chain.

1.5 Objectives

ID	Objective
OBJ-001	Design and implement a differential-drive miniature AGV with encoder feedback, IMU integration, ultrasonic obstacle detection, a
OBJ-002	Develop a structured UART communication protocol between the ESP32 (embedded control layer) and the Raspberry Pi 5 (edge
OBJ-003	Establish an MQTT-based publish/subscribe telemetry and command infrastructure hosted on the Raspberry Pi 5, with defined to
OBJ-004	Implement a Digital Twin engine on the Raspberry Pi 5 that maintains a continuously synchronised virtual representation of the p
OBJ-005	Deploy a web-based supervisory dashboard that provides real-time AGV visualisation, mission management, analytical charts, a
OBJ-006	Define and execute a structured experimental research plan to measure physical-to-digital synchronisation latency, command lat
OBJ-007	Design a modular, extensible software architecture that supports future multi-AGV coordination, SLAM-based localisation, and cl
OBJ-008	Implement safety mechanisms including hardware emergency stop, software watchdog, motor-shutdown-on-obstacle, and comm

1.6 Scope

In Scope:

- Miniature warehouse physical setup (approximately 1.5 m × 1.5 m to 2 m × 2 m tabletop/floor model)
- Single-AGV differential-drive platform with encoder and IMU feedback
- ESP32 real-time firmware (motor control, sensor acquisition, safety, UART comms)
- Raspberry Pi 5 edge node (MQTT broker, telemetry processor, navigation logic, API backend, database)
- MQTT-based bidirectional communication architecture
- Digital Twin engine with state synchronisation, historical data, and analytics
- Web-based supervisory dashboard
- Structured mission management (create, assign, execute, complete, cancel)
- Performance experiment plan for synchronisation latency, localisation, and resource utilisation
- Safety mechanisms: emergency stop, obstacle halt, battery protection, watchdog
- Project documentation and academic research paper preparation support

Out of Scope (Base Version):

- Multi-AGV fleet coordination (noted as future extension)
- Full SLAM or LiDAR-based mapping (noted as advanced optional extension)
- Cloud-hosted digital twin infrastructure
- Industrial safety certification (IEC 61508, ISO 3691-4)
- AI-based predictive maintenance (noted as future work)
- Computer vision-based object recognition beyond fiducial markers
- Production-grade cybersecurity hardening
- Outdoor or unstructured environment operation

1.7 Target Users

User Role	Responsibilities
System Administrator	Manages Raspberry Pi services, MQTT broker configuration, database maintenance, and system health mo
Warehouse Operator	Monitors AGV position and status, creates and manages missions, responds to alerts via the dashboard.
Research Engineer / Student	Designs and conducts experiments, analyses logged data, measures performance KPIs, prepares research
Maintenance Operator	Diagnoses faults, replaces hardware components, updates firmware, reviews diagnostic logs.
Project Evaluator / Academic Supervisor	Assesses system functionality, reviews SRS compliance, evaluates demonstration scenarios and research c

1.8 Definitions and Acronyms

Acronym / Term	Definition
AGV	Autonomous Guided Vehicle — a mobile robotic platform capable of self-guided navigation within a structured environm
CPS	Cyber-Physical System — a system integrating computation, networking, and physical processes.
DT	Digital Twin — a dynamic virtual representation of a physical entity updated from real-world sensor data.
MQTT	Message Queuing Telemetry Transport — a lightweight publish/subscribe messaging protocol designed for IoT applicat
UART	Universal Asynchronous Receiver-Transmitter — a serial communication protocol used for ESP32↔RPi data exchange
PWM	Pulse Width Modulation — a technique for encoding an analogue signal as a digital pulse train, used here for motor spe
IMU	Inertial Measurement Unit — a sensor module combining accelerometer and gyroscope for motion state estimation.
PID	Proportional-Integral-Derivative — a feedback control algorithm used for closed-loop motor speed and heading control.
SLAM	Simultaneous Localisation and Mapping — a computational technique enabling a robot to build a map while tracking its
IoT	Internet of Things — a network of physical devices embedded with sensors and software enabling data exchange.
QoS	Quality of Service — in MQTT, QoS levels (0, 1, 2) define the delivery guarantee semantics for messages.
API	Application Programming Interface — a structured interface for software components to communicate.
OTA	Over-The-Air — refers to remote firmware update capability.
RPi	Raspberry Pi — specifically Raspberry Pi 5 in this document unless otherwise stated.
RPM	Revolutions Per Minute — unit of rotational speed for motor/wheel measurement.
BMS	Battery Management System — electronics for monitoring and protecting a battery pack.
ToF	Time-of-Flight — a proximity/distance sensing technology based on measuring round-trip time of light pulses.

SRS	Software Requirements Specification — this document.
FR	Functional Requirement — a requirement specifying system behaviour.
NFR	Non-Functional Requirement — a requirement specifying system quality attributes.
KPI	Key Performance Indicator — a measurable metric used to evaluate system performance.
RFID	Radio Frequency Identification — used for station detection in marker-based localisation.
ROS2	Robot Operating System 2 — an optional advanced middleware for robot software development.
LAN	Local Area Network — the local wireless/wired network over which the system communicates.
REST	Representational State Transfer — an architectural style for web APIs using HTTP verbs.

2. System Overview

The CyberTwin-AGV system is an integrated cyber-physical framework comprising five hierarchically organised layers. Each layer has well-defined responsibilities, interfaces, and communication contracts. The complete information flow is described below.

2.1 Conceptual Architecture Flow

Layer	Component	Role	Interface to Next Layer
Physical (L1)	Warehouse, AGV chassis, motors, wheels, sensors	Physical actuation and sensing	Electrical signals → ESP32
Embedded Control (L2)	ESP32 microcontroller	Real-time control, safety, sensor fusion	UART → Raspberry Pi 5
Edge Computing (L3)	Raspberry Pi 5	MQTT, backend, database, DT sync, navigation	MQTT / REST / WebSocket → DT + Dashbo
Digital Twin (L4)	DT engine + virtual warehouse	State mirroring, analytics, historical data	REST / WebSocket → Dashboard
Application (L5)	Web Dashboard	Operator visualisation, mission control, diagnostics	HTTP commands → Backend → MQTT →

2.2 Information Flow Description

Physical → Embedded (L1 → L2): Sensors (encoders, IMU, ultrasonic, battery monitor) provide raw electrical signals to the ESP32 via GPIO, ADC, I2C, and digital interrupt pins. Motors and their drivers receive PWM and direction signals from the ESP32.

Embedded → Edge (L2 → L3): The ESP32 serialises sensor data and state information into a structured JSON message and transmits it to the Raspberry Pi 5 over a UART serial link at a configurable baud rate (proposed: 115200 bps). The Raspberry Pi transmits command messages to the ESP32 over the same UART link.

Edge → Digital Twin and Backend (L3 → L4): The Raspberry Pi 5 hosts an MQTT broker (Mosquitto). A telemetry processing service subscribes to MQTT topics, parses incoming messages, stores data to the database, and calls the Digital Twin synchronisation engine to update the virtual AGV state. A FastAPI REST backend exposes APIs for the dashboard to query current state, historical data, and mission records.

Digital Twin → Application (L4 → L5): The dashboard communicates with the FastAPI backend over HTTP/REST and WebSocket for live state streaming. Operators interact with the dashboard to view real-time state, create missions, and issue commands.

Application → Physical (L5 → L1): Commands issued from the dashboard are sent via REST POST to the backend, published as MQTT command messages, received by the telemetry bridge on the Raspberry Pi, forwarded over UART to the ESP32, and executed by the motor control loop.

2.3 Key Design Principles

- **Edge-First Processing:** All primary data processing and digital twin synchronisation occur locally on the Raspberry Pi 5. Cloud dependency is not required for core operation.
- **Bidirectional Synchronisation:** The system supports both physical-to-digital (telemetry reporting) and digital-to-physical (command execution) information flows.

• **Modular Architecture:** Each software component (MQTT bridge, DT engine, API backend, navigation module, database) is independently deployable and replaceable.

• **Research Instrumentation:** All critical communication paths and processing stages are designed with timestamp capture to enable latency measurement experiments.

• **Fault Tolerance:** Communication failures trigger well-defined safe states at both the embedded and edge layers.

• **Scalability:** The MQTT topic structure and backend API design support extension to multi-AGV operation without architectural change.

3. System Architecture

3.1 Layer 1 — Physical Layer

The Physical Layer constitutes all tangible hardware components of the miniature warehouse environment and the AGV platform. This layer is the sole source of ground-truth physical state.

Sub-Component	Description
Miniature Warehouse Structure	A rigid platform (plywood, acrylic, or equivalent) demarcating storage racks, stations, AGV lanes, charging z
AGV Chassis	A wheeled robotic platform capable of differential-drive motion. Constructed from aluminium plate, acrylic, or
Drive Motors	Two brushed DC geared motors providing independent left and right wheel drive for differential steering.
Wheels and Tyres	Rubber-tyred wheels providing traction on the warehouse floor surface.
Motor Encoders	Incremental quadrature encoders (or hall-effect sensors) co-located with each drive motor for closed-loop sp
Inertial Measurement Unit (IMU)	A combined accelerometer-gyroscope module (e.g., MPU-6050 / ICM-42688 class) providing angular velocit
Ultrasonic / ToF Distance Sensors	Forward-facing and optionally side-facing proximity sensors for obstacle detection in the AGV's path.
Battery Pack	A rechargeable LiPo or Li-ion battery pack powering the AGV electronics and motors.
Battery Voltage / Current Sensor	An INA219 or equivalent sensor monitoring battery state of charge and power consumption.
Optional: Camera Module	A Raspberry Pi Camera Module or USB webcam for AprilTag detection or overhead visual localisation.
Optional: AprilTag / QR Markers	Fiducial markers placed at defined positions within the warehouse for absolute localisation correction.

3.2 Layer 2 — Embedded Control Layer (ESP32)

The ESP32 microcontroller is the real-time embedded control node. It operates independently of the Raspberry Pi for all time-critical control functions, ensuring that safety and motor control remain functional even if the edge node experiences a software fault.

Module	Responsibility
Motor PWM Generation	Generates PWM signals for both motor driver channels to control motor speed and direction independently.
Encoder Interrupt Handling	Processes quadrature encoder pulses via hardware interrupts to compute real-time wheel speed and cumula
IMU Data Acquisition	Reads IMU registers over I2C at a defined rate (proposed: 50–100 Hz) and applies a complementary or Kalr
Obstacle Sensor Polling	Triggers ultrasonic sensors and measures echo pulse widths. Alternatively reads ToF sensor distance via I2
Battery Monitoring	Reads battery voltage and current via ADC and INA219 to monitor state of charge and power consumption.
PID Control Loop	Implements closed-loop PID controllers for left and right wheel speed independently, using encoder feedbac
UART Communication	Serialises telemetry into JSON-structured messages and transmits at a defined rate. Receives and parses c
Safety Manager	Monitors obstacle distance, battery level, heartbeat timeout, and motor fault conditions. Triggers emergency
Command Parser	Decodes command messages from the Raspberry Pi (e.g., SET_VELOCITY, STOP, SET_MISSION) and di
Watchdog Timer	Hardware watchdog that resets the ESP32 if the main control loop stalls for longer than a defined threshold

3.3 Layer 3 — Edge Computing Layer (Raspberry Pi 5)

The Raspberry Pi 5 acts as the on-premises edge computing hub. It runs all higher-level software services and hosts the Digital Twin synchronisation engine.

Service / Module	Responsibility
UART-MQTT Bridge	Reads incoming telemetry from the ESP32 over UART, adds ingestion timestamps, and publishes to MQ
MQTT Broker (Mosquitto)	Hosts the Mosquitto broker for the local MQTT pub/sub infrastructure. All AGV telemetry flows through th
Telemetry Processor	Subscribes to MQTT telemetry topics, validates and parses messages, and writes to the database.
Navigation Service	Maintains the AGV's estimated position using odometry (encoder + IMU). Provides waypoint sequencing
Mission Manager	Manages the mission lifecycle: creation, queuing, assignment, execution tracking, completion, and cance
Digital Twin Sync Engine	Maintains the virtual AGV model. Updates the DT state from incoming telemetry. Serves DT state to the
FastAPI REST Backend	Exposes REST and WebSocket endpoints for dashboard communication. Provides AGV status, telemetr
InfluxDB / SQLite Database	Time-series storage (InfluxDB) for sensor telemetry and relational storage (SQLite) for mission records,
Logging Service	Structured logging of all system events, errors, and state transitions to rotating log files.
Health Monitor	Monitors Raspberry Pi CPU, memory, disk, and service availability. Publishes diagnostics to MQTT.

3.4 Layer 4 — Digital Twin Layer

The Digital Twin layer constitutes the virtual representation of the physical AGV and warehouse. It is not a passive dashboard — it actively maintains a stateful model of the system and supports bidirectional interaction.

Component	Description
Virtual Warehouse Model	A 2D map representation of the warehouse including station positions, storage racks, lanes, charging zon
Virtual AGV Model	A stateful object representing the AGV with attributes: position (x, y, θ), velocity, wheel RPM, battery level,
State Synchronisation Engine	Processes incoming telemetry and updates the virtual AGV position, orientation, and status. Calculates sy
Event Store	Records all state transitions, fault events, mission starts/completions, and alert triggers with timestamps fo
Historical Data Access	Provides time-range queries over stored telemetry for trend visualisation, performance analysis, and resea
Command Gateway	Accepts validated commands from the application layer and routes them to the MQTT command topics for
Anomaly Detection (Basic)	Detects anomalous sensor readings (e.g., unexpected battery voltage drop, encoder count mismatch) and

3.5 Layer 5 — Application Layer (Dashboard)

The application layer provides the human-facing interface for the complete system. It is implemented as a web application accessible via a browser on any device connected to the same local network as the Raspberry Pi.

Screen	Content and Functionality
Overview Dashboard	Real-time warehouse map with AGV position overlay, status indicators, battery gauge, active mission sum

AGV Details View	Detailed telemetry panel: speed, wheel RPM, IMU data, battery voltage/current, obstacle distances, comm
Mission Management Panel	Interface to create, queue, start, pause, and cancel AGV missions. Displays mission history with completio
Analytics Dashboard	Time-series charts for speed, battery, temperature, and latency. Mission duration histograms, error rate tre
Diagnostics Panel	System health view: MQTT broker status, database status, Raspberry Pi CPU/memory/temperature, servi
Alert and Log Viewer	Chronological log of all system events, alerts, faults, and state transitions. Filterable by severity and categ

4. Hardware Requirements

This section defines the hardware Bill of Materials (BOM) for the CyberTwin-AGV system. Component specifications marked as *proposed* represent the team's current recommendation and are subject to revision based on availability and budget. Final component selection must be validated against the GPIO mapping and power budget before procurement.

4.1 Computing — Edge Node

Component	Qty	Purpose	Interface	Proposed Spec	Selection Rationale
Raspberry Pi 5	1	Edge computing node; hosts all	GPIO, USB, UART, Ethernet, Wi-Fi	8 GB RAM, BCM2712 quad-core	32/16 bit CPU, 2.4 GHz quad-core over RPi 4; P
RPi 5 Official PSU	1	Stable 5.1 V / 5 A power supply	USB-C	27 W rated	Required to prevent under-voltage at full load
Active Cooler (RPi 5)	1	Thermal management under sustained load	GPIO (fan header)	Official RPi active cooler	Prevents thermal limits under sustained
MicroSD Card	1	OS and storage	SD/MicroSD	64 GB+ Class 10 / A2 UHS-I	A2 rating ensures sufficient random-read IOP
Optional: NVMe SSD (M.2 HAT)	1	Higher IOPS storage for InfluxDB	PCIe via M.2 HAT	128 GB M.2 NVMe	Recommended if InfluxDB write throughput e

4.2 Embedded Microcontroller

Component	Qty	Purpose	Interface	Proposed Spec	Selection Rationale
ESP32 Development Board	1	Real-time embedded control layer	GPIO, UART, I2C, SPI, ADC, PWM, Wi-Fi, BT	ESP32-WROOM-32 (Xtensa LX6 dual-core, 240 MHz, 520 KB SRAM, 4 MB flash)	Low-cost, high-performance MCU with built-in
USB-UART Adapter (backup)	1	Flashing and debug via PC	USB	CP2102 or CH340G based	Enables firmware flashing and serial monitor

4.3 AGV Mechanical and Drive System

Component	Qty	Purpose	Interface	Proposed Spec	Selection Rationale
Robot Chassis	1	Structural platform for AGV	—	Two-layer acrylic or aluminum, ~250 × 180 mm	Compact size; supports differential-drive layout
DC Geared Motor with Encoder	2	Drive wheels; provide encoder feedback	PWM, GPIO, Interrupt	12 V, 100–200 RPM at output shaft, 300 CPR encoder	Reduces speed and increases
Motor Driver Module	1	Amplifies PWM signals to drive motors	PWM, Direction GPIO from ESP32	L298N or TB6612FNG (dual H-bridge, 2 A per channel)	Dual-bridge, 2 A supports direction reversal; ad
Rubber Wheels	2	Traction on warehouse floors	Press-fit on motor shaft	65 mm diameter, rubber tyre	Provides grip; standard fit for selected motor
Caster Wheel	1	Passive support wheel (rear/front)	Mechanical	Ball caster or swivel caster	Provides stability without constraining differe
Chassis Standoffs and Hardware	—	Mechanical assembly	—	M3 nuts, bolts, standoffs	Structural assembly

4.4 Sensor Suite

Component	Qty	Purpose	Interface	Proposed Spec	Category
IMU Module	1	Orientation and angular velocity	I2C (ESP32)	MPU-6050 or ICM-42688-P (6-axis)	Mandatory
Ultrasonic Distance Sensor	2–3	Obstacle detection (front, optional rear)	GPIO (TRIG, ECHO)	HC-SR04, range: 2–400 cm, resolution: ~3 mm	Mandatory
ToF Distance Sensor (alt.)	2–3	Obstacle detection alternative to ultrasonic	I2C (ESP32)	VL53L1X, range: up to 4 m, 50 Hz update	Proposed alternative

Battery Voltage/Current Sensor	1	Battery SoC monitoring and power draw	I2C (ESP32)	INA219 (26 V / 3.2 A range)	Mandatory
Line Sensor Array (optional)	1	Lane guidance for Level 1 navigation	GPIO/ADC (ESP32)	5-channel IR line sensor	Optional for Level 1 nav
Camera Module (optional)	1	AprilTag detection or visual localisation	CSI or USB (RPi)	Raspberry Pi Camera v2 or USB webcam	Optional for Level 3 nav
RFID Reader (optional)	1	Station identification	SPI or UART (ESP32)	RC522 or equivalent	Optional
AprilTag Markers (optional)	—	Absolute position reference	Visual (camera)	Printed A5 markers (Tag36h11 family)	Optional for Level 3 nav

4.5 Power System

Component	Qty	Purpose	Interface	Proposed Spec	Notes
LiPo Battery Pack	1	Primary AGV power supply	Direct / via BMS	7.4 V (2S) or 11.1 V (3S), 2200–5000 mAh	Provides runtime of approximately 1–2 h
Battery Management System	1	Overcharge, over-discharge, short-circuit protection	In-line with battery	Matched to cell count and capacity	Strongly recommended for LiPo safety
Buck Converter (5 V)	1	Regulated 5 V rail for ESP32 and sensors	Input from battery, output to 5 V bus	LM2596 or MP1584 module, ≥ 3 A	Isolates logic power from motor noise
Buck Converter (3.3 V, if needed)	1	3.3 V for sensors requiring lower supply	Input from 5 V rail or battery	AMS1117-3.3 or similar	Optional if all sensors operate at 5 V
Power Switch	1	System power on/off	In-line with battery positive	Rated for total system current draw	Mandatory for safe operation
Emergency Stop Button	1	Physical hardware emergency stop	GPIO interrupt (ESP32)	Normally-closed pushbutton, latching	Safety-critical; see Section 17
Fuse / Resettable Fuse	1	Short-circuit protection	In-line with battery	Rated for maximum expected current	Safety requirement
Charging Connector	1	Battery recharging without disassembly	External access on chassis	XT30 or JST-XH balance connector	Enables docking station simulation

4.6 Warehouse Physical Structure

Component	Qty	Purpose	Notes
Warehouse Base Platform	1	Defines AGV operating area boundary	Plywood sheet, foam board, or rigid floor mat; ~1.5–2 m × 1.5–
Storage Rack Models	4–6	Represent warehouse storage locations	3D-printed or cardboard boxes labelled A–F
Station Markers (AprilTag/RFID)	4–6	Machine-readable station identification	Printed AprilTag or RFID tag at each station position
AGV Lane Markings	—	Visual lane guidance for Level 1 navigation	White tape or printed lane lines on the base platform
Charging Zone	1	Designated AGV recharging area	Marked area with charging connector or simulated contact pad
Safety Boundary Markers	—	Define no-go zones for the AGV	Physical barriers or coloured tape at warehouse edges
Cable Management	—	Route power and data cables safely	Cable ties, conduit, labelled connectors

4.7 Optional Advanced Components

The following components are explicitly marked as OPTIONAL. They extend the capability of the base system but are not required for MVP completion. Their inclusion depends on budget, team capacity, and academic timeline.

Component	Purpose	Why Optional
2D LiDAR (e.g., RPLIDAR A1)	360° obstacle detection and SLAM-based mapping	High cost; complex ROS2 integration; not required for pred

Raspberry Pi Camera v3 (wide angle)	AprilTag visual localisation at higher accuracy	Camera-based localisation adds significant image processi
Load Cell / Cargo Sensor	Detect if AGV is carrying cargo	Nice-to-have for realistic mission simulation but not archite
OLED / LCD Status Display on AGV	Local status display without dashboard	Useful for debugging but not required in final system
Wi-Fi Range Extender	Extend local network coverage in larger warehouse	Required only if warehouse exceeds typical RPi Wi-Fi rang
Raspberry Pi UPS HAT	Uninterruptible power supply for RPi 5	Prevents data corruption on sudden power loss; recommer

5. Hardware Interfaces

This section defines the hardware interface specifications for all communication buses and signal lines used in the CyberTwin-AGV system. All pin assignments listed are proposed and must be verified against the selected ESP32 development board variant before PCB layout or firmware implementation.

5.1 ESP32 Interface Overview

Interface	Connected Peripheral	Direction	Protocol / Signal	Notes
UART0 (TX0/RX0)	USB for firmware flash/debug	Bidirectional	UART 115200 bps	Used during development only
UART2 (TX2/RX2)	Raspberry Pi 5 UART	Bidirectional	UART 115200 bps (configurable)	Primary telemetry and command channel; 3.3 V logic;
I2C (SDA/SCL)	IMU (MPU-6050/ICM-42688) + INA219	Master	I2C, 400 kHz fast mode	Pull-up resistors on SDA/SCL required
GPIO (Digital Output)	Motor Driver (PWM + DIR) × 2 channels	Output	PWM (LEDC) + Digital	Motor A: GPIO12, GPIO13; Motor B: GPIO14, GPIO27
GPIO (Digital Input)	Encoder A/B pulses (×2 motors)	Input (interrupt)	Digital interrupt	Encoder L_A: GPIO34, L_B: GPIO35; R_A: GPIO36, R
GPIO (Digital I/O)	Ultrasonic sensors TRIG/ECHO (×2)	TRIG: Output; ECHO: Input	Digital pulse	Front: TRIG=GPIO5, ECHO=GPIO18; Side: TRIG=GP
ADC	Battery voltage divider (if INA219 not used)	Input (ADC)	Analogue 0–3.3 V	ADC1_CH6 = GPIO34 (if dedicated to battery voltage)
GPIO (Digital Input)	Emergency stop button	Input (interrupt)	Digital, active-low, pull-up	GPIO0 or dedicated pin; triggers immediate motor shu

5.2 Proposed ESP32 GPIO Pin Mapping

The following is a proposed pin assignment for an ESP32-WROOM-32 / ESP32-DevKitC board. GPIO capabilities vary between ESP32 variants. Pins GPIO34–GPIO39 are input-only on standard ESP32. Final assignments must be cross-referenced against the selected board's pinout diagram.

GPIO	Assigned Function	Direction	Interface	Notes
GPIO12	Motor A — PWM (Left Motor Speed)	Output	LEDC PWM	Connect to ENA / PWMA on motor driver
GPIO13	Motor A — DIR (Left Motor Direction)	Output	Digital	Connect to IN1 on motor driver
GPIO14	Motor B — PWM (Right Motor Speed)	Output	LEDC PWM	Connect to ENB / PWMB on motor driver
GPIO27	Motor B — DIR (Right Motor Direction)	Output	Digital	Connect to IN3 on motor driver
GPIO34	Left Encoder — Channel A	Input (interrupt)	Digital pulse	Input-only GPIO; internal pull-up not available
GPIO35	Left Encoder — Channel B	Input (interrupt)	Digital pulse	Input-only GPIO
GPIO36 (VP)	Right Encoder — Channel A	Input (interrupt)	Digital pulse	Input-only GPIO
GPIO39 (VN)	Right Encoder — Channel B	Input (interrupt)	Digital pulse	Input-only GPIO
GPIO21 (SDA)	I2C SDA — IMU + INA219 + ToF	Bidirectional	I2C	4.7 kΩ pull-up to 3.3 V required
GPIO22 (SCL)	I2C SCL — IMU + INA219 + ToF	Output (open-drain)	I2C	4.7 kΩ pull-up to 3.3 V required
GPIO16 (RX2)	UART2 RX — From Raspberry Pi TX	Input	UART	Level-shift if RPi GPIO is 3.3 V (usually compatible)
GPIO17 (TX2)	UART2 TX — To Raspberry Pi RX	Output	UART	
GPIO5	Ultrasonic Front TRIG	Output	Digital pulse	10 µs trigger pulse
GPIO18	Ultrasonic Front ECHO	Input	Digital pulse	Measure echo pulse width

GPIO19	Ultrasonic Side TRIG (optional)	Output	Digital pulse	
GPIO23	Ultrasonic Side ECHO (optional)	Input	Digital pulse	
GPIO0	Emergency Stop Button	Input (interrupt)	Active-low	External pull-up; BOOT pin on some boards — verify compatibi
GPIO2	Status LED (onboard/debug)	Output	Digital	Optional visual indicator

5.3 Raspberry Pi 5 Interface Connections

RPi 5 Pin / Interface	Connected To	Protocol	Notes
GPIO14 (TXD) — Pin 8	ESP32 UART2 RX (GPIO16)	UART 3.3 V	Telemetry and command serial link
GPIO15 (RXD) — Pin 10	ESP32 UART2 TX (GPIO17)	UART 3.3 V	Telemetry receive from ESP32
GND — Pin 6 or 14	ESP32 GND	Reference	Common ground required
USB Port	USB hub, keyboard (setup only)	USB	For initial OS configuration
Ethernet Port	Local network switch/router	Gigabit Ethernet	Preferred for stable MQTT; Wi-Fi also supported
CSI Camera Port	Raspberry Pi Camera Module (Pi CSI-2)	MIPI CSI-2	For AprilTag detection
Wi-Fi (internal)	Local 2.4/5 GHz Wi-Fi network	IEEE 802.11 b/g/n/ac	Dashboard and MQTT accessibility
USB-C Power	Official RPi 5 PSU	5.1 V / 5 A	Power input

5.4 Communication Bus Electrical Specifications

Bus	Voltage Level	Speed / Rate	Max Cable Length (approx.)	Notes
UART (ESP32 ↔ RPi)	3.3 V logic	115200 bps (proposed)	≤ 1 m (on chassis)	Use twisted pair or shielded wire; common ground ess
I2C (ESP32 peripherals)	3.3 V logic	400 kHz (Fast Mode)	≤ 0.5 m recommended	Add 4.7 kΩ pull-ups; keep wires short
PWM (ESP32 → Motor Driver)	3.3 V logic signal	1–20 kHz carrier	≤ 0.3 m	Motor driver input typically compatible with 3.3 V logic
Motor Power (Driver → Motors)	Battery voltage (7.4–11.1 V)	N/A	≤ 0.5 m	Use appropriately rated wire; separate from logic wirin
USB (RPi — peripherals)	5 V	USB 2.0 / 3.0	≤ 3 m standard	For camera, input devices during setup
Ethernet (RPi — Network)	N/A	1 Gbps	≤ 100 m (Cat5e)	Preferred over Wi-Fi for lower latency

6. Embedded Firmware Requirements

The ESP32 firmware implements all real-time control and safety functions. It is structured as a modular, FreeRTOS-based application where each functional module runs as an independent task or set of functions with defined priorities and timing constraints.

FW-MOD-01 — Motor Controller Module	
Purpose	Generates PWM signals on the appropriate GPIO channels to drive the left and right motor channels of the H-bridge mo
Inputs	Target velocity commands (m/s) for each wheel; direction flag; enable flag.
Outputs	PWM duty cycle (0–100%) on motor PWM GPIO; digital direction signal on DIR GPIO.
Responsibilities	• Maps target velocity to PWM duty cycle using calibrated motor characterisation curve. • Implements ramping to avoid sudden large speed changes. • Supports forward, reverse, and stationary states. • Applies hard stop (zero duty cycle, coasting or braking) on STOP command.
Failure Condition	If motor driver reports fault (if fault pin available), set duty cycle to 0 and raise MOTOR_FAULT event.

FW-MOD-02 — Encoder Manager Module	
Purpose	Acquires wheel encoder pulses via hardware interrupts and computes real-time wheel speed and cumulative displacem
Inputs	Encoder A/B channel pulses (interrupt-driven); elapsed time from FreeRTOS tick counter.
Outputs	Left and right wheel RPM; left and right encoder tick count (cumulative).
Responsibilities	• Attaches interrupt service routines (ISR) to encoder GPIO pins. • Counts pulses per unit time to compute RPM. • Accumulates total tick count for odometry computation. • Handles quadrature decoding (direction-aware counting) if quadrature encoders are used.
Failure Condition	If encoder count does not change while motor is commanded to move, raise ENCODER_STALL event.

FW-MOD-03 — IMU Manager Module	
Purpose	Reads IMU registers over I2C at a defined sample rate and applies a sensor fusion filter to produce orientation estimate
Inputs	Raw accelerometer (ax, ay, az) and gyroscope (gx, gy, gz) registers from IMU over I2C.
Outputs	Filtered pitch, roll, yaw angles (radians or degrees); angular velocity; linear acceleration.
Responsibilities	• Initialises IMU with appropriate full-scale range and DLPF settings. • Reads sensor data at 50–100 Hz. • Applies complementary filter (or Kalman filter) to fuse accel and gyro. • Detects IMU communication failure (I2C NAK) and raises IMU_FAULT.
Failure Condition	On I2C communication failure, set IMU status flag to FAULT and continue with degraded localisation.

FW-MOD-04 — Obstacle Sensor Manager	
Purpose	Manages ultrasonic or ToF sensors to measure distances in the AGV's travel path.
Inputs	Echo pulse widths from ultrasonic ECHO GPIO pins (or I2C distance from ToF sensors).
Outputs	Obstacle distance (cm or mm) for each sensor direction; OBSTACLE_DETECTED boolean flag.

Responsibilities	• Triggers ultrasonic sensors sequentially to avoid cross-interference. • Converts echo time to distance using speed of sound. • Compares measured distance against configurable threshold (proposed: 15–30 cm). • Sets OBSTACLE_DETECTED flag and raises OBSTACLE_EVENT if threshold exceeded. • Clears flag when path is unobstructed for a defined debounce period.
Failure Condition	If sensor returns out-of-range values consistently, flag as SENSOR_FAULT; continue with remaining sensors.

FW-MOD-05 — Battery Monitor Module

Purpose	Monitors battery voltage and current to estimate state of charge and detect low-battery conditions.
Inputs	Voltage and current readings from INA219 over I2C (or ADC voltage divider reading).
Outputs	Battery voltage (V); current (A); estimated SoC (%); BATTERY_LOW and BATTERY_CRITICAL flags.
Responsibilities	• Reads INA219 at a low rate (e.g., 1–2 Hz to reduce overhead). • Computes estimated SoC from voltage-SoC lookup table for the selected cell chemistry. • Raises BATTERY_LOW alert when SoC < configurable threshold (e.g., 20%). • Raises BATTERY_CRITICAL and initiates return-to-charging-station command when SoC < 5%.
Failure Condition	If INA219 I2C communication fails, flag as BATT_SENSOR_FAULT; use last known voltage.

FW-MOD-06 — PID Controller Module

Purpose	Implements independent closed-loop PID velocity controllers for left and right wheels using encoder feedback.
Inputs	Target wheel speed (RPM); measured wheel speed from Encoder Manager.
Outputs	Corrected PWM duty cycle for each wheel passed to Motor Controller.
Responsibilities	• Computes error (target - measured) at each control loop iteration. • Accumulates integral term with anti-windup clamping. • Computes derivative term with low-pass filtering to reduce noise sensitivity. • Applies output limits to prevent exceeding safe duty cycle range. • Supports runtime tuning of Kp, Ki, Kd parameters via UART command.
Failure Condition	If output saturates persistently, raise PID_WINDUP warning and log.

FW-MOD-07 — UART Communication Manager

Purpose	Manages bidirectional structured communication between the ESP32 and the Raspberry Pi 5 over UART2.
Inputs	Telemetry data from all sensor and control modules; incoming command bytes from Raspberry Pi.
Outputs	Serialised JSON telemetry messages transmitted at a defined rate; parsed command structures dispatched to Comman
Responsibilities	• Assembles JSON telemetry message from latest sensor/state data at configurable rate (proposed: 10 Hz). • Transmits message with newline delimiter for easy parsing. • Receives bytes from Raspberry Pi and buffers into complete message. • Validates received message (JSON parse; CRC or length check if implemented). • Updates heartbeat receive timestamp; triggers communication timeout if no message received for threshold period.
Failure Condition	If no valid message received from RPi for > configurable timeout (e.g., 5 seconds), trigger COMMS_TIMEOUT safe stat

FW-MOD-08 — Safety Manager Module

Purpose	Central safety monitoring module that enforces safe operating conditions and triggers emergency stop when required.

Inputs	OBSTACLE_DETECTED flag; BATTERY_CRITICAL flag; COMMS_TIMEOUT flag; emergency stop GPIO state; watch
Outputs	EMERGENCY_STOP command to Motor Controller; SAFE_STATE flag; safety event log entries.
Responsibilities	• Continuously monitors all safety input flags. • Triggers immediate motor stop on OBSTACLE_DETECTED with distance < hard-stop threshold (e.g., 10 cm). • Triggers motor stop and BATTERY_CRITICAL_STOP on BATTERY_CRITICAL. • Triggers motor stop on COMMS_TIMEOUT (heartbeat lost from Raspberry Pi). • Responds to emergency stop button GPIO interrupt within < 50 ms. • Prevents motor restart until safety condition is resolved and restart is explicitly authorised.
Failure Condition	This module must not itself fail silently. Watchdog ensures module execution continues.

FW-MOD-09 — Diagnostics Manager Module

Purpose	Tracks module health, error counts, and system runtime metrics and includes them in telemetry messages.
Inputs	Error flags and event counts from all modules; FreeRTOS task runtime stats.
Outputs	Diagnostics payload appended to UART telemetry; local LED status indication.
Responsibilities	• Counts occurrences of each error event type. • Records last-error timestamp for each module. • Reports FreeRTOS task high-water stack usage to detect stack overflow risk. • Sets status LED patterns to indicate system health (e.g., green: ok; amber: warning; red: fault).
Failure Condition	Non-critical; degraded diagnostics should not affect motor control or safety modules.

FW-MOD-10 — Command Parser Module

Purpose	Parses structured command messages received from the Raspberry Pi via UART and dispatches them to the appropria
Inputs	Validated JSON command message from UART Communication Manager.
Outputs	Dispatched control commands to Motor Controller / PID / Safety Manager / Mission state.
Responsibilities	• Recognises command types: SET_VELOCITY, SET_HEADING, STOP, RESUME, EMERGENCY_STOP, SET_PID, I • Validates command parameters (range checks, authorisation token if implemented). • Responds with acknowledgement message over UART. • Logs unknown or malformed commands as COMMAND_ERROR events.
Failure Condition	Invalid commands are rejected with error response. Parser failure does not affect ongoing motor control.

6.1 Firmware Task Priority Table

FreeRTOS Task	Priority (0=lowest)	Period / Trigger	Core Affinity	Notes
Safety Monitor Task	10 (highest)	Continuous / 10 ms	Core 1	Must never be preempted by lower-priority tasks during
PID Control Task	8	10 ms (100 Hz)	Core 1	Tight timing required for stable velocity control
Encoder ISR	Hardware interrupt	On encoder pulse	Both	ISR posts to queue; PID task reads
Obstacle Sensor Task	7	50 ms (20 Hz)	Core 0	Polls ultrasonic sensors sequentially
IMU Acquisition Task	7	10–20 ms (50–100 Hz)	Core 0	I2C read + filter computation
UART Comm Task	6	100 ms (10 Hz) TX; continuous RX	Core 0	Assembles and transmits telemetry; receives comman
Battery Monitor Task	4	500 ms (2 Hz)	Core 0	Low frequency; non-time-critical

Diagnostics Task	2	1000 ms (1 Hz)	Core 0	Collects stats; lowest priority
Watchdog Feed	9	Every 500 ms	Core 1	Resets hardware watchdog; if task hangs, WDT trigge

7. AGV Control System

7.1 Differential Drive Kinematics

The AGV uses a differential-drive configuration with two independently driven wheels and one passive caster. The kinematic equations governing the relationship between wheel velocities and robot motion are:

Parameter	Symbol	Expression	Notes
Linear velocity (robot)	v	v = (v_R + v_L) / 2	v_R, v_L = right/left wheel velocities
Angular velocity (robot)	ω	ω = (v_R – v_L) / L	L = wheel track (distance between drive wheels)
Left wheel velocity	v_L	v_L = (RPM_L × π × D) / 60	D = wheel diameter
Right wheel velocity	v_R	v_R = (RPM_R × π × D) / 60	D = wheel diameter
Wheel circumference	C	C = π × D	D = wheel diameter (m)
Displacement per tick	d_tick	d_tick = C / PPR	PPR = pulses per revolution (encoder)

These equations are implemented in both the ESP32 PID module (for wheel velocity control) and the Raspberry Pi navigation service (for odometry position estimation).

7.2 Motor Control and PWM

Motor speed is controlled via PWM signals generated by the ESP32's LEDC (LED Control) peripheral, which provides hardware PWM with configurable frequency and resolution. A typical configuration uses a PWM frequency of 1–20 kHz with 8-bit (0–255) or 10-bit (0–1023) resolution. The motor driver (L298N or TB6612FNG) accepts PWM and direction signals and drives the motors at the proportional voltage.

7.3 Encoder-Based Speed Measurement

Wheel encoders produce pulse trains proportional to wheel rotation. The encoder ISR increments a counter on each rising edge (or each quadrature transition for direction-aware counting). The PID control task reads the counter value at a fixed interval (e.g., every 10 ms) and computes instantaneous RPM:

```
RPM = (ΔTicks / ΔTime_seconds) × (60 / PPR)
```

where ΔTicks is the encoder count difference since last measurement and PPR is the encoder's pulses per revolution.

7.4 Odometry

Dead-reckoning odometry estimates the AGV's position (x, y) and heading (θ) by integrating encoder-derived wheel displacements over time. The update equations at each timestep Δt are:

```
Δs = (Δs_R + Δs_L) / 2 [forward displacement] Δθ = (Δs_R – Δs_L) / L [heading change]
x_new = x + Δs × cos(θ + Δθ/2) y_new = y + Δs × sin(θ + Δθ/2) θ_new = θ + Δθ
```

Odometry is executed on the Raspberry Pi 5 navigation service using encoder data received via MQTT telemetry. IMU heading data provides complementary angular correction to reduce gyroscopic drift.

7.5 PID Velocity Control

Independent PID controllers maintain the left and right wheel speeds at their commanded setpoints. The PID update is executed at 100 Hz (10 ms period):

```
error(t) = setpoint(t) − measured(t) u(t) = Kp × error(t) + Ki × ∫error dt + Kd × d(error)/dt PWM_duty = clip(u(t), 0, PWM_MAX)
```

Anti-windup is applied to the integral term by clamping the accumulated integral when the output is saturated. Initial PID tuning will be performed empirically using a step-response methodology.

7.6 Emergency Stop

The emergency stop mechanism is implemented at two levels:

- **Hardware Emergency Stop:** A normally-closed pushbutton in the motor power circuit physically interrupts motor power when pressed. Simultaneously, the GPIO interrupt on the ESP32 sets the EMERGENCY_STOP flag and disables PWM outputs.
- **Software Emergency Stop:** Any system component (Safety Manager, Obstacle Manager, Communication Timeout handler, or Raspberry Pi command) can issue an EMERGENCY_STOP command. The Safety Manager immediately sets motor PWM to 0 and enters the SAFE_STOPPED state. The system remains in SAFE_STOPPED until an explicit RESUME command with cleared safety conditions is received.

7.7 Obstacle Detection and Response

Distance Threshold	Response	Recovery
< 30 cm (warning)	Reduce speed to 50% of setpoint; publish OBSTACLE_WARNING via MQTT	Automatic when distance > 30 cm
< 15 cm (soft stop)	Stop motors; publish OBSTACLE_STOP via MQTT; enter OBSTACLE_WAIT state	Resume command if distance > 20 cm
< 10 cm (hard stop)	Immediate motor stop (same as emergency stop logic); publish OBSTACLE_CRITICAL	Requires explicit RESUME after clearance

Threshold values are configurable via UART SET_PARAM command and stored in ESP32 NVS (Non-Volatile Storage) so that they persist across reboots.

7.8 Autonomous Navigation Modes

Mode	Description	Who Controls
MANUAL	Operator sends continuous velocity commands from dashboard via MQTT → ESP32	Human operator via dashboard
SEMI-AUTONOMOUS	Operator specifies a waypoint or direction; AGV navigates to it independently using PID and odometry	Operator + Raspberry Pi navigation service
AUTONOMOUS MISSION	Mission Manager assigns a full mission (source station → destination station). AGV autonomously executes navigation	Mission Manager + Raspberry Pi

8. Localization and Navigation

Navigation capability is defined in four staged levels, ordered by complexity and accuracy. The project team should implement Level 1 as the baseline MVP, and progressively implement higher levels as time and resources permit.

8.1 Level 1 — Predefined Paths / Waypoints

Description: The AGV follows hardcoded sequences of motor commands corresponding to predefined routes between warehouse stations. Navigation is open-loop (no feedback correction). Optionally, a line-sensor array may provide lane-following feedback.

Implementation: Mission Manager sends a sequence of (velocity, duration) or (velocity, encoder_target) commands to the ESP32 via MQTT. The ESP32 executes each segment and reports completion. Station arrival may be detected by elapsed encoder ticks, timer expiry, or RFID reader trigger.

> ***Accuracy:*** *Position accuracy is dependent on floor surface, wheel slip, and battery voltage variation. Suitable for demonstration but not quantitatively precise.*

8.2 Level 2 — Encoder + IMU Odometry

Description: The Raspberry Pi 5 navigation service computes the AGV's estimated position (x, y, θ) using encoder-derived wheel displacements and IMU-derived heading. The navigation service generates wheel velocity setpoints for each leg of a route based on the current estimated position and the next waypoint.

Coordinate System: A 2D Cartesian coordinate system with origin at a defined reference point (e.g., warehouse corner or charging station). Units: metres. Heading is measured from positive X-axis, increasing counter-clockwise.

> ***Limitation:*** *Odometry drift accumulates over time due to wheel slip, encoder quantisation, and IMU gyroscope bias. Positional error will grow with distance travelled. Periodic absolute position correction (Level 3) is recommended.*

8.3 Level 3 — Marker-Based Absolute Localisation

Description: AprilTag or QR code markers are placed at known positions within the warehouse (at each station, at intersections, or at the charging zone). When the AGV's camera detects a marker, its position and orientation relative to the marker are computed using computer vision (e.g., OpenCV with AprilTag library). The navigation service uses this absolute position fix to correct the odometry estimate.

Detection: The Raspberry Pi camera captures frames and runs the detection algorithm as a background service. Detected markers include a tag ID (mapped to warehouse position) and a 6-DoF pose estimate.

> ***RFID Alternative:*** *If camera-based detection is impractical, RFID tags at each station and an RFID reader on the AGV can provide binary station-arrival confirmation, enabling simple absolute position reset at known points.*

8.4 Level 4 (Optional) — LiDAR-Based Localisation / SLAM

SLAM (Simultaneous Localisation and Mapping) using a 2D LiDAR sensor enables the AGV to build a map of the warehouse and localise within it without pre-placed markers. This level is explicitly OPTIONAL and should only be attempted if: (a) a LiDAR sensor is available, (b) the team has capacity to integrate ROS2 or an equivalent SLAM library, and (c) the academic timeline permits. SLAM integration significantly

increases software complexity and is not required for MVP completion.

8.5 Path Planning and Waypoint Sequencing

Concept	Description
Waypoint	A defined (x, y, θ) pose in the warehouse coordinate system representing an intermediate or final navigation ta
Path	An ordered sequence of waypoints from the AGV's current position to the mission destination, avoiding no-go z
Path Planning (Level 1–2)	For small, structured warehouses with predefined routes, paths are defined statically as waypoint lists in config
Path Planning (Level 3–4)	Grid-based or graph-based search algorithms (e.g., A*, Dijkstra) compute optimal paths dynamically based on
Obstacle Avoidance (Dynamic)	During mission execution, if an obstacle is detected, the navigation service may: (a) wait for clearance, (b) com
Navigation Error	Defined as the Euclidean distance between the AGV's estimated position at station arrival and the ground-truth

9. Communication Architecture

9.1 UART Communication (ESP32 ↔ Raspberry Pi 5)

The primary physical communication link between the embedded control layer and the edge computing layer is a UART serial link. This provides a deterministic, low-overhead communication channel independent of network infrastructure.

Parameter	Value / Description
Baud Rate	115200 bps (proposed; configurable at compile time)
Data Bits	8
Stop Bits	1
Parity	None
Flow Control	None (software framing used)
Message Format	JSON-encoded string, newline (\n) terminated
Telemetry Direction	ESP32 → Raspberry Pi (at 10 Hz proposed rate)
Command Direction	Raspberry Pi → ESP32 (event-driven; also heartbeat at 1 Hz)
Message Size (typical)	200–500 bytes per telemetry message
Throughput at 10 Hz	~5 KB/s maximum; well within UART capacity at 115200 bps
Error Detection	JSON parse validation; optional CRC field in message structure

9.1.1 ESP32 Telemetry Message Structure

The following JSON structure defines the proposed telemetry message transmitted by the ESP32 at each reporting interval:

```
{ "msg_type": "TELEMETRY", "seq": 12345, "ts_ms": 1692345678000, "agv_id": "AGV-001",
"motion": { "rpm_left": 87.3, "rpm_right": 86.9, "v_linear": 0.28, "v_angular": 0.01 },
"imu": { "pitch_deg": 0.3, "roll_deg": -0.1, "yaw_deg": 45.2, "ax": 0.01, "ay": 0.00,
"az": 9.81 }, "obstacles": { "front_cm": 45.2, "left_cm": 120.0, "right_cm": 98.5,
"detected": false }, "battery": { "voltage_v": 7.82, "current_a": 0.54, "soc_pct": 73 },
"status": { "state": "MOVING", "estop": false, "comms_ok": true, "error_flags": 0 },
"diag": { "uptime_s": 3600, "cpu_temp_c": 38, "free_heap": 152340 } }
```

9.1.2 Raspberry Pi Command Message Structure

```
{ "msg_type": "COMMAND", "seq": 567, "ts_ms": 1692345680000, "cmd": "SET_VELOCITY",
"params": { "v_linear": 0.30, "v_angular": 0.0 }, "auth": "HEARTBEAT_TOKEN_ABC" }
```

Valid command types include: SET_VELOCITY, STOP, EMERGENCY_STOP, RESUME, SET_PID, SET_PARAM, MISSION_START, MISSION_ABORT, PING, RESET_ODOMETRY.

9.2 MQTT Communication (Raspberry Pi ↔ Digital Twin ↔ Dashboard)

MQTT (Message Queuing Telemetry Transport) provides the publish/subscribe communication backbone for the edge computing, digital twin, and application layers. The Mosquitto MQTT broker runs as a system

service on the Raspberry Pi 5.

Parameter	Configuration
Broker Software	Eclipse Mosquitto v2.x
Broker Host	Raspberry Pi 5 (localhost or LAN IP)
Default Port	1883 (unencrypted local LAN); 8883 (TLS, if configured)
Protocol Version	MQTT v3.1.1
Authentication	Username/password (local access); certificate-based TLS optional
Retained Messages	Used for last-known-good state topics (e.g., status, position)
Last Will and Testament (LWT)	AGV publishes LWT on connect; broker publishes on disconnect
Heartbeat	AGV heartbeat published on twinware/agv/heartbeat at 1 Hz
Keep-Alive	60 seconds (MQTT PINGREQ/PINGRESP mechanism)
Reconnection	Clients implement exponential back-off reconnection (max 60 s)

9.3 MQTT Topic Hierarchy

Topic	Publisher	Subscriber(s)	QoS	Retained	Purpose
twinware/agv/{id}/telemetry	UART-MQTT Bridge (RPi)	DT Sync Engine, DB Writer	1	No	Full sensor telemetry at 10 Hz
twinware/agv/{id}/status	DT Sync Engine (RPi)	Dashboard, Backend API	1	Yes	AGV operational state summary
twinware/agv/{id}/position	Navigation Service (RPi)	DT Sync Engine, Dashboard	1	Yes	Estimated (x, y, θ) position
twinware/agv/{id}/command	Mission Manager / Dashboard	UART-MQTT Bridge (RPi)	2	No	Motor/navigation commands to AGV
twinware/agv/{id}/mission	Mission Manager (RPi)	DT Sync Engine, Dashboard	1	Yes	Current mission state and assignment
twinware/agv/{id}/alert	Safety Mgr / Battery Mgr (RPi)	Dashboard, Backend API	2	No	Safety alerts and critical events
twinware/agv/{id}/diagnostics	Health Monitor (RPi)	Dashboard	0	No	RPi CPU, memory, service status (1 Hz)
twinware/agv/{id}/heartbeat	UART-MQTT Bridge (RPi)	DT Sync Engine, Dashboard	0	No	Liveness pulse (1 Hz)
twinware/agv/{id}/loglevel	Dashboard / Admin	All RPi services	1	Yes	Runtime log-level control
twinware/warehouse/map	Backend (RPi)	Dashboard	1	Yes	Static warehouse map data
twinware/system/status	Health Monitor (RPi)	Dashboard	1	Yes	Overall system health summary
twinware/agv/{id}/lwt	Broker (on disconnect)	DT Sync Engine, Dashboard	1	Yes	Last Will: AGV considered offline

Note: {id} is the AGV identifier string (e.g., 'agv-001'). For a single-AGV system, this is fixed. The topic structure supports multi-AGV extension by varying {id}.

9.4 QoS Level Selection Rationale

QoS Level	Semantics	Use Cases in This Project
QoS 0 (At most once)	Fire-and-forget; no acknowledgement; possible loss	Heartbeat, diagnostics — high frequency, loss tolerable
QoS 1 (At least once)	Acknowledged delivery; possible duplicates	Telemetry, status, position — loss not tolerable but duplicates acceptable

QoS 2 (Exactly once)	Guaranteed exactly-once delivery; highest overhead	Commands, alerts — must not be lost or duplicated

10. Digital Twin Requirements

This section defines what constitutes the Digital Twin in this project and distinguishes it from a simple telemetry dashboard. The Digital Twin is a **stateful, continuously synchronised virtual representation** of the physical AGV and warehouse, with bidirectional interaction capability, historical data access, and analytical functionality.

10.1 Digital Twin vs. Simple Dashboard

Capability	Simple Dashboard	Digital Twin (This Project)
Live sensor display	Yes	Yes
Stateful virtual model	No	Yes — maintains AGV state object
Historical data access	Limited / none	Yes — time-series database
Bidirectional commands	No	Yes — commands propagate to physical AGV
Mission state tracking	No	Yes — full mission lifecycle management
Synchronisation latency measurement	No	Yes — timestamps tracked at all layers
Event and fault history	No	Yes — event store with severity and timestamps
Anomaly detection	No	Yes (basic) — unexpected reading detection
Analytics and trend charts	No	Yes — speed, battery, latency trends
Virtual warehouse map	No	Yes — 2D map with AGV position overlay

10.2 Digital Twin States

State	Description	Entry Condition	Exit Condition
ONLINE_IDLE	AGV connected; no active mission; awaiting assignment	Valid heartbeat received; no mission assigned	Mission assigned or manual control started
ONLINE_MOVING	AGV connected; actively moving on a mission or manual command	Mission executing; velocity command active	Mission complete, stop command, or obstacle detected
ONLINE_CHARGING	AGV at charging station; battery replenishment in progress	AGV in charging zone AND SoC < threshold	SoC reaches full or departure command issued
OBSTACLE_HALT	AGV stopped due to obstacle detected in path	Obstacle distance < halt threshold	Path cleared and RESUME command issued
EMERGENCY_STOPPED	Emergency stop triggered (hardware or software)	E-stop button pressed or EMERGENCY_STOP command received	EMCY_RESET command issued and safety checks passed
MISSION_ACTIVE	Mission in progress; AGV moving between waypoints	Mission assigned and execution started	Mission complete, failed, or cancelled
DEGRADED	AGV connected but operating with one or more faults active	Sensor fault flagged in telemetry	Fault resolved or maintenance performed
OFFLINE	No heartbeat received for > timeout threshold	Heartbeat missing for > 30 s (configurable)	Heartbeat resumes; transition to ONLINE_IDLE
STALE	Telemetry received but older than synchronisation threshold	Last telemetry timestamp > configurable staleness threshold	Fresh telemetry received

10.3 Synchronisation Requirements

Synchronisation latency is defined as the elapsed time from the moment a physical state change occurs on the AGV to the moment the Digital Twin's virtual model reflects that change. The measurement chain is:

```
Physical event occurs → ESP32 reads sensor (latency: sensor polling interval ≤ 20 ms) → ESP32 assembles and transmits UART message (latency: ≤ 100 ms at 10 Hz rate) → Raspberry Pi receives and parses UART message (latency: ≤ 5 ms) → Raspberry Pi publishes MQTT message (latency: ≤ 5 ms on localhost broker) → DT Sync Engine receives MQTT and updates virtual model (latency: ≤ 10 ms) → Dashboard WebSocket update sent to client (latency: ≤ 50 ms over LAN)
■■■
```

11. Digital Twin Data Model

The Digital Twin data model defines the structured entities that constitute the virtual representation of the AGV and warehouse. These entities are maintained in memory by the DT Sync Engine and persisted to the database.

11.1 AGV Entity

Field Name	Type	Description	Notes
agv_id	String	Unique AGV identifier (e.g., 'AGV-001')	Immutable
timestamp	DateTime (UTC)	Timestamp of the most recently applied telemetry update	Auto-updated
position_x	Float (m)	Estimated x-coordinate in warehouse coordinate frame	From navigation service
position_y	Float (m)	Estimated y-coordinate in warehouse coordinate frame	From navigation service
orientation_deg	Float (°)	Estimated heading angle (0–360°)	From IMU + odometry
rpm_left	Float (RPM)	Left wheel rotational speed	From encoder
rpm_right	Float (RPM)	Right wheel rotational speed	From encoder
v_linear	Float (m/s)	Computed linear velocity	From kinematic model
v_angular	Float (rad/s)	Computed angular velocity	From kinematic model
battery_voltage	Float (V)	Measured battery terminal voltage	From INA219
battery_current	Float (A)	Measured battery draw current	From INA219
battery_soc_pct	Integer (0–100)	Estimated state of charge percentage	From SoC estimator
state	Enum	Current DT state (see Section 10.2)	From DT Sync Engine
mission_id	String / None	ID of currently assigned mission	From Mission Manager
obstacle_front_cm	Float (cm)	Measured front obstacle distance	From ultrasonic/ToF
estop_active	Boolean	True if emergency stop is active	From safety status
comms_status	Enum	Communication state (CONNECTED/DEGRADED/OFFLINE)	From DT Sync Engine
error_flags	Integer (bitmask)	Bitmask of active ESP32 error flags	From telemetry
firmware_version	String	ESP32 firmware version string	From diagnostics message
last_sync_latency_ms	Float (ms)	Measured sync latency for last telemetry update	Computed by DT Engine

11.2 Sensor Reading Entity

Field Name	Type	Description	Notes
reading_id	UUID	Unique identifier for this reading	Auto-generated
agv_id	String	Foreign key to AGV entity	
sensor_id	String	Sensor identifier (e.g., 'IMU-01', 'ULTRA-FRONT')	

sensor_type	Enum	Type: IMU / ULTRASONIC / TOF / BATTERY / ENCODER	
value	Float or JSON	Sensor measured value (or JSON for multi-dimensional)	
unit	String	Unit of measurement (m/s, °, V, A, cm, RPM, etc.)	
timestamp	DateTime (UTC)	Timestamp of sensor measurement on ESP32	From UART message
ingestion_timestamp	DateTime (UTC)	Timestamp of receipt on Raspberry Pi	Added by bridge
status	Enum	Reading validity: VALID / SENSOR_FAULT / OUT_OF_RANGE	

11.3 Mission Entity

Field Name	Type	Description	Notes
mission_id	UUID	Unique mission identifier	Auto-generated
agv_id	String	Foreign key to AGV entity	
source_station	String	Origin station identifier (e.g., 'STATION-A')	Operator input
destination_station	String	Target station identifier	Operator input
priority	Integer (1–5)	Mission priority (1 = highest)	Operator input, default 3
created_at	DateTime (UTC)	Timestamp of mission creation	Auto-generated
assigned_at	DateTime (UTC)	Timestamp of mission assignment to AGV	Set by Mission Manager
started_at	DateTime (UTC)	Timestamp when AGV began executing mission	Set by Mission Manager
completed_at	DateTime (UTC)	Timestamp of mission completion or failure	Set by Mission Manager
status	Enum	CREATED / QUEUED / ASSIGNED / EXECUTING / PAUSED / COMPLETED / FAILED / CA	
failure_reason	String / None	Reason for failure if status = FAILED	
duration_seconds	Float / None	Elapsed time from started_at to completed_at	Computed
distance_travelled_m	Float / None	Estimated distance covered during mission	From odometry
obstacle_events	Integer	Number of obstacle halts during this mission	Counted by Mission Mgr
operator_id	String	Identifier of operator who created the mission	From auth session

11.4 Event Entity

Field Name	Type	Description	Notes
event_id	UUID	Unique event identifier	Auto-generated
agv_id	String	Foreign key to AGV entity	
event_type	Enum	Type: FAULT / ALERT / INFO / STATE_CHANGE / COMMAND / MISSION	
severity	Enum	CRITICAL / HIGH / MEDIUM / LOW / INFO	
timestamp	DateTime (UTC)	Time of event occurrence	
source	String	Originating module (e.g., 'SafetyManager', 'MissionManager')	

description	String	Human-readable event description	
acknowledged	Boolean	Whether operator has acknowledged the event	Default False
acknowledged_at	DateTime / None	Timestamp of acknowledgement	
metadata	JSON / None	Additional event-specific structured data	

12. Database Requirements

12.1 Database Technology Selection

Criterion	SQLite	InfluxDB	PostgreSQL
Type	Relational (SQL)	Time-series	Relational (SQL)
Deployment complexity	Zero-config, file-based	Service with Flux/SQL query	Service, more resource-intensive
Best for	Missions, events, configuration, users	High-frequency sensor telemetry	Complex relational queries, multi-table joins
Raspberry Pi suitability	Excellent	Good (watch memory on 4 GB)	Good with tuning
Query language	SQL	Flux / InfluxQL	SQL
Recommended use	Mission records, events, user data, system config	Telemetry time-series (10 Hz sensor data)	Optional, if complex analytics needed

*Proposed architecture: Use **InfluxDB** for all telemetry time-series data (high write frequency, time-range queries) and **SQLite** for missions, events, system configuration, and user records (structured relational data, lower write frequency). This dual-database approach balances performance with deployment simplicity.*

12.2 InfluxDB — Telemetry Bucket Schema

Measurement	Tags	Fields	Retention Policy
agv_motion	agv_id, session_id	rpm_left, rpm_right, v_linear, v_angular	30 days (configurable)
agv_imu	agv_id, session_id	pitch, roll, yaw, ax, ay, az	30 days
agv_battery	agv_id, session_id	voltage_v, current_a, soc_pct	90 days
agv_obstacles	agv_id, session_id	front_cm, left_cm, right_cm, detected	30 days
agv_position	agv_id, session_id	x_m, y_m, theta_deg, nav_level	30 days
agv_latency	agv_id, session_id	uart_latency_ms, mqtt_latency_ms, dt_sync_ms, e2e_ms	90 days
system_health	node_id	cpu_pct, mem_pct, disk_pct, temp_c	7 days

12.3 SQLite — Relational Schema (Key Tables)

Table Name	Primary Key	Key Columns	Purpose
agv	agv_id (TEXT)	firmware_version, mac_address, registration_date, description	AGV registry
missions	mission_id (UUID)	agv_id, source_station, destination_station, status, created_at, completed_at, operator_id	Mission lifecycle records
events	event_id (UUID)	agv_id, event_type, severity, timestamp, source, description, acknowledged	System event log
waypoints	waypoint_id (UUID)	name, x_m, y_m, theta_deg, station_type, description	Warehouse waypoint definitions
users	user_id (UUID)	username, role, password_hash, created_at, last_login	Dashboard user accounts
system_config	config_key (TEXT)	config_value, data_type, description, last_modified	Runtime configuration parameters
alerts	alert_id (UUID)	agv_id, alert_type, severity, raised_at, cleared_at, message	Active and historical alerts

firmware_versions	version_id (UUID)	agv_id, version_string, deployed_at, notes	Firmware deployment history

12.4 Timestamp Requirements

All database records must include timestamps in UTC (Coordinated Universal Time) to ensure consistency across system components. The Raspberry Pi 5 must maintain time synchronisation via NTP (Network Time Protocol). Telemetry messages include two timestamps: (a) the ESP32 millisecond uptime counter (relative, for latency computation) and (b) the Raspberry Pi UTC ingestion timestamp (absolute, for database storage and cross-referencing).

13. Backend Requirements

The backend is a Python-based REST and WebSocket API server running on the Raspberry Pi 5. The proposed technology is FastAPI, chosen for its async support, automatic OpenAPI documentation generation, and compatibility with Python-based ecosystem components.

13.1 Backend Services

AGV Status Service: Returns the current state of the Digital Twin (position, velocity, battery, state, mission, sensor summary). Served from in-memory DT model.

Mission API Service: CRUD operations on missions: create, list, get by ID, cancel, pause, resume. Validates mission parameters (valid station IDs, AGV availability).

Telemetry Query Service: Queries InfluxDB for time-range telemetry data. Supports filtering by measurement type and time window. Returns data for dashboard charting.

Event and Alert Service: Lists events and alerts by AGV, time range, severity. Supports alert acknowledgement. Returns event count summaries for dashboard.

Command Service: Validates and publishes commands to MQTT command topics. Requires authenticated session. Returns command acknowledgement status.

DT Synchronisation Engine: Internal service (not directly exposed via REST). Subscribes to MQTT telemetry, updates in-memory DT model, computes synchronisation latency, writes to database.

Health Monitoring Service: Collects Raspberry Pi system metrics (CPU, memory, temperature, disk) and publishes to MQTT diagnostics topic and database.

WebSocket Broadcast Service: Maintains WebSocket connections with dashboard clients. Pushes real-time DT state updates when the DT model changes.

13.2 REST API Endpoints (Conceptual)

The following endpoints are defined conceptually. Implementation details (request/response schemas, authentication middleware) are deferred to the detailed design phase.

Method	Endpoint	Description
GET	/api/v1/agv/{id}/status	Current DT state summary for specified AGV
GET	/api/v1/agv/{id}/telemetry	Historical telemetry with time-range and field filter params
GET	/api/v1/agv/{id}/position	Current and historical position estimates
POST	/api/v1/command	Issue a command to the AGV (validated and authorised)
GET	/api/v1/missions	List all missions (supports filter by status, date range)
POST	/api/v1/missions	Create a new mission
GET	/api/v1/missions/{id}	Get mission details by ID
PUT	/api/v1/missions/{id}/cancel	Cancel a queued or executing mission
PUT	/api/v1/missions/{id}/pause	Pause an executing mission
PUT	/api/v1/missions/{id}/resume	Resume a paused mission
GET	/api/v1/events	List events (filter by type, severity, time range)

PUT	/api/v1/events/{id}/ack	Acknowledge an event
GET	/api/v1/alerts	List active and historical alerts
GET	/api/v1/diagnostics	Raspberry Pi system health metrics
GET	/api/v1/warehouse/map	Warehouse layout and waypoint definitions
POST	/api/v1/auth/login	Authenticate user; return session token
POST	/api/v1/auth/logout	Invalidate session token
GET	/api/v1/health	Backend liveness check
WebSocket	/ws/live	Real-time DT state push stream to dashboard

14. Digital Twin Dashboard Requirements

The dashboard is a web-based application served by the Raspberry Pi 5 and accessible via any browser on the local network. It provides the primary human interface for monitoring, control, and analysis.

DASH-01 — Overview Dashboard

Primary landing screen providing at-a-glance system status.

- 2D warehouse map with AGV position indicator (updated via WebSocket, ≤ 200 ms latency)
- AGV state badge (IDLE / MOVING / OBSTACLE / E-STOP / OFFLINE — colour-coded)
- Battery level gauge (percentage with colour: green > 50%, amber 20–50%, red < 20%)
- Active mission summary card (mission ID, source, destination, elapsed time, progress)
- Active alert panel (sorted by severity; newest first)
- System health indicators (MQTT status, DB status, backend status — green/red)
- Connection quality indicator (last telemetry timestamp; latency estimate)

DASH-02 — AGV Details View

Detailed real-time telemetry display for the AGV.

- Speed gauge (linear velocity in m/s; max scale configurable)
- Left and right wheel RPM bar charts
- IMU orientation display (pitch, roll, yaw as numerical values and orientation graphic)
- Battery voltage/current trend mini-charts (last 60 seconds)
- Obstacle sensor distance indicators (front, left, right with visual proximity bar)
- Communication status (UART OK, MQTT OK, last message age)
- Firmware version and ESP32 uptime
- Error flags panel (binary indicator for each fault category)

DASH-03 — Mission Management Panel

Interface for creating, monitoring, and controlling missions.

- Mission creation form: source station selector, destination station selector, priority picker
- Submit / Clear form controls
- Mission queue list (queued missions with position indicator)
- Active mission tracker (real-time status with estimated completion)
- Mission history table (all past missions with status, duration, distance, obstacles encountered)
- Mission controls: Pause, Resume, Cancel buttons (state-appropriate enabling)
- Mission statistics summary (total missions, success rate, average duration)

DASH-04 — Analytics Dashboard

Time-series charts and performance analytics.

- Speed vs. time chart (selectable time range: last 5 min, 30 min, 1 h, 24 h)
- Battery SoC over time chart

- Synchronisation latency histogram and trend (physical-to-digital)
- MQTT message rate chart (messages per second)
- Packet loss rate indicator
- Mission duration distribution histogram
- Obstacle event frequency over time
- Raspberry Pi CPU and memory usage trend
- Data export button (download CSV for selected time range and measurement)

DASH-05 — Diagnostics Panel

System health and service status display.

- MQTT broker status (connected clients, message rate, uptime)
- InfluxDB status (write rate, disk usage, query response time)
- SQLite status (size, last write time)
- FastAPI backend status (uptime, request rate, error rate)
- Raspberry Pi metrics (CPU per core, total RAM usage, temperature, disk free)
- DT Sync Engine status (sync rate, last sync latency, queue depth)
- Navigation service status (active/inactive, current navigation level)
- Service restart controls (admin role only)

DASH-06 — Event Log and Alert Viewer

Chronological audit trail of all system events.

- Chronological event table (event ID, timestamp, type, severity, source, description)
- Filter controls (by severity, type, date range, AGV ID)
- Alert acknowledge button per unacknowledged alert
- Export log as CSV
- Real-time streaming of new events via WebSocket

15. Warehouse Model

15.1 Physical Warehouse Layout

The miniature warehouse is a structured tabletop or floor-level environment designed to simulate a simplified warehouse operation. The layout is defined by a 2D coordinate system and includes the following zones:

Zone	Description	Coordinate Example
Station A (Loading)	Primary cargo loading station; AGV picks up from here	(0.3, 0.3) m
Station B	Secondary storage station	(1.2, 0.3) m
Station C	Tertiary storage station	(1.5, 1.0) m
Station D	Delivery / unloading station	(0.3, 1.5) m
Charging Zone	AGV returns here for battery recharging simulation	(0.0, 0.0) m (origin)
AGV Lane (Horizontal)	Primary horizontal travel corridor	y = 0.8 m, x = 0.0 to 1.8 m
AGV Lane (Vertical)	Primary vertical travel corridor	x = 0.8 m, y = 0.0 to 1.8 m
Safety Boundary	Physical perimeter; AGV must not travel beyond	x = [0.0, 2.0], y = [0.0, 2.0] m
No-Go Zone	Areas occupied by physical obstacles/rack structures	Defined as rectangular exclusion zones in ma

15.2 Example Warehouse Mission Workflow

Step	Action	AGV State	Notes
1	AGV at Charging Zone; idle	ONLINE_IDLE	Battery at 100%; awaiting mission
2	Operator creates mission: Station A → Station D	ONLINE_IDLE	Mission entered via dashboard
3	Mission Manager queues and assigns mission	MISSION_ACTIVE	AGV transitions to ASSIGNED state
4	Navigation service computes route: Charging → Lane H → Station A	MOVING	Waypoint sequence generated
5	AGV travels to Station A (cargo pick-up)	MOVING	Encoder + odometry tracking
6	AGV arrives at Station A; mission pauses for loading	IDLE (at station)	Station arrival confirmed (RFID/marker/timer)
7	Obstacle detected mid-route to Station D	OBSTACLE_HALT	Obstacle distance < halt threshold
8	Obstacle cleared; AGV resumes	MOVING	RESUME command from navigation service
9	AGV arrives at Station D; mission complete	ONLINE_IDLE	Mission record updated: COMPLETED
10	Battery SoC < 20%; return to charging mission auto-created	MOVING → CHARGING	Automatic low-battery return mission

16. Mission Management

16.1 Mission Lifecycle States

State	Description	Entry Condition	Valid Transitions
CREATED	Mission record exists; not yet queued	Operator submits mission via dashboard	→ QUEUED, → CANCELLED
QUEUED	Mission waiting in queue for AGV availability	All AGVs busy or mission priority < active mission	→ ASSIGNED, → CANCELLED
ASSIGNED	Mission allocated to specific AGV; navigation computation in progress	AGV becomes available; Mission Manager assigns	→ EXECUTING, → CANCELLED
EXECUTING	AGV actively navigating to destination	Route computed; first velocity command sent	→ PAUSED, → COMPLETED, → FAILED, → C
PAUSED	Mission temporarily suspended (obstacle, operator request)	Obstacle detected or pause command received	→ EXECUTING (RESUME), → CANCELLED, –
COMPLETED	AGV successfully reached destination and delivered cargo	AGV arrives at destination station	Terminal state
FAILED	Mission could not be completed (fault, timeout, battery)	Safety fault, path timeout, or irrecoverable obstacle	Terminal state
CANCELLED	Mission explicitly cancelled by operator or system	Operator cancel or system override	Terminal state

16.2 Mission Manager Responsibilities

- Validate mission parameters before accepting (valid station IDs, AGV not in fault state).
- Queue missions and apply priority-based scheduling (highest priority executed first).
- Assign queued missions to the AGV when it becomes available.
- Coordinate with Navigation Service to compute and execute the route.
- Monitor mission progress via telemetry (position updates, obstacle events).
- Update mission state in SQLite database at each state transition.
- Publish mission state changes to MQTT mission topic.
- Automatically create return-to-charging mission when battery SoC falls below critical threshold.
- Detect and handle mission timeouts (mission in EXECUTING state for longer than configurable maximum).
- Log all state transitions and failures to the event store.

17. Safety Requirements

■ IMPORTANT DISCLAIMER: All safety mechanisms described in this section are PROTOTYPE-GRADE implementations intended for a controlled laboratory/academic environment. This system has NOT been evaluated, certified, or approved under any industrial safety standard (including but not limited to IEC 61508, ISO 3691-4, or EN ISO 13849). This prototype must NEVER be operated in environments with uncontrolled human access. Physical barriers must be in place during operation.

SAF-001 — Hardware Emergency Stop	
Requirement	A physically accessible, normally-closed emergency stop button shall be installed on the AGV chassis. Pressing
Implementation Guidance	Hardware: NC button in series with motor driver enable line. Software: GPIO interrupt triggers Safety Manager E
Verification	TC-SAF-001: Press e-stop button while AGV is in motion; verify motor stop within 50 ms.

SAF-002 — Software Emergency Stop Command	
Requirement	The system shall support a software-initiated emergency stop command from the dashboard or MQTT comman
Implementation Guidance	Dashboard POST /api/v1/command with cmd=EMERGENCY_STOP → MQTT QoS 2 → UART bridge → ESP3
Verification	TC-SAF-002: Issue e-stop from dashboard; verify motor halt and EMERGENCY_STOPPED state in DT.

SAF-003 — Obstacle-Triggered Motor Halt	
Requirement	If any obstacle sensor reports a distance below the configured hard-stop threshold, the Safety Manager shall im
Implementation Guidance	Obstacle Manager sets OBSTACLE_DETECTED flag; Safety Manager polls at 100 Hz; sets motor PWM to 0.
Verification	TC-SAF-003: Place obstacle in AGV path; verify halt before contact.

SAF-004 — Communication Timeout Safe State	
Requirement	If the ESP32 does not receive a valid heartbeat from the Raspberry Pi for longer than a configurable timeout (de
Implementation Guidance	UART Comm Manager tracks last heartbeat receive timestamp; Safety Manager checks at 1 Hz.
Verification	TC-SAF-004: Terminate RPi UART bridge service; verify ESP32 stops motors within timeout.

SAF-005 — Battery Critical Protection	
Requirement	When battery SoC falls below the critical threshold (proposed: 5%), the system shall halt current mission, stop m
Implementation Guidance	Battery Monitor raises BATTERY_CRITICAL flag; Safety Manager responds immediately.
Verification	TC-SAF-005: Simulate low battery voltage via parameter command; verify mission abort and motor halt.

SAF-006 — Hardware Watchdog Timer	
Requirement	The ESP32 hardware watchdog timer shall be configured to reset the microcontroller if the main control loop sta
Implementation Guidance	FreeRTOS watchdog task feeds the hardware WDT every 500 ms. If any critical task hangs, the WDT fires.
Verification	TC-SAF-006: Simulate main loop hang via debug injection; verify ESP32 resets within 2 seconds.

SAF-007 — Maximum Velocity Limit

Requirement	The AGV's maximum operational linear velocity shall be configurable and shall default to 0.5 m/s. The motor co
Implementation Guidance	Motor Controller clamps all velocity setpoints to MAX_VELOCITY parameter before PID computation.
Verification	TC-SAF-007: Send velocity command exceeding limit; verify clamped execution and logged warning.

SAF-008 — Safe Startup Sequence

Requirement	On power-up, the AGV shall perform a self-test sequence: initialise all sensors, verify communication, and only
Implementation Guidance	ESP32 main() performs sequential module initialisation and reports INIT_OK or INIT_FAIL per module before en
Verification	TC-SAF-008: Power cycle AGV; verify startup sequence in UART log; verify no motor activation during init.

SAF-009 — Physical Safety Boundaries

Requirement	The warehouse design shall include physical barriers preventing the AGV from leaving the defined operating are
Implementation Guidance	Navigation service checks estimated position against warehouse boundary polygon at each update; issues BOU
Verification	TC-SAF-009: Command AGV toward warehouse boundary; verify warning and halt before boundary contact.

SAF-010 — Safe Shutdown Procedure

Requirement	The system shall support a graceful shutdown command that: stops the AGV, saves state to database, closes d
Implementation Guidance	Dashboard admin button or systemd stop command triggers graceful shutdown sequence on RPi.
Verification	TC-SAF-010: Initiate graceful shutdown; verify all services stop, data saved, no corruption.

18. Security Requirements

Security requirements for this prototype are scoped appropriately for an academic research system operating on a local network. Production-grade hardening (TLS everywhere, PKI certificate management, formal threat modelling) is not required for the base project but should be considered for any deployment beyond the laboratory environment.

ID	Requirement	Priority
SEC-001	The MQTT broker shall require username and password authentication for all clients. Default/blank credentials shall not be perm	High
SEC-002	Dashboard access shall require a username and password login. Session tokens shall expire after a configurable idle period (de	High
SEC-003	The FastAPI backend shall validate and authenticate all API requests using session tokens. Unauthenticated requests to non-pu	High
SEC-004	Command API endpoints (e.g., POST /api/v1/command) shall require operator or administrator role. Read-only endpoints may be	High
SEC-005	MQTT topics shall be access-controlled: only authorised publishers may publish to command topics (twin/agv/{id}/command	Medium
SEC-006	Credentials (MQTT password, database passwords, API secret keys) shall not be hardcoded in source code. They shall be store	High
SEC-007	The UART communication channel between ESP32 and Raspberry Pi shall include a simple shared token in command message	Medium
SEC-008	All system access and command events shall be logged with timestamps and user identifiers to the event store for auditability.	Medium
SEC-009	The project repository shall not contain any real credentials, API keys, or sensitive configuration values. A .env.example template	High
SEC-010	Firmware update capability (OTA) is considered an optional advanced feature. If implemented, firmware binaries shall be integrity	Low (optional)
SEC-011	The Raspberry Pi OS shall be kept updated with security patches during the development period. Default SSH passwords shall b	Medium
SEC-012	The Raspberry Pi shall not expose unnecessary network services. Only MQTT (1883), FastAPI (8000/8080), and dashboard (80/	Medium

19. Non-Functional Requirements Overview

Non-functional requirements define the quality attributes of the system. They constrain how the system behaves rather than what it does. The following categories are defined for the CyberTwin-AGV system:

Category	Description	Key Concerns
Performance	System response times, throughput, latency	Synchronisation latency, command latency, sensor rate
Reliability	Fault tolerance, availability, recovery	MQTT reconnection, watchdog, fault handling
Scalability	Ability to handle growth	Multi-AGV extension, data volume growth
Maintainability	Ease of modification and debugging	Modular code, logging, documentation
Usability	Ease of use for operators	Dashboard clarity, alert management
Portability	Migration to different hardware	Docker containers, documented configuration
Security	Protection from unauthorised access	Authentication, authorisation, credential management
Testability	Ability to verify requirements experimentally	Measurable metrics, log instrumentation
Safety	Protection from harm	Emergency stop, obstacle halt, watchdog

20. Performance Requirements and KPIs

The following Key Performance Indicators (KPIs) define measurable targets for the system. All values marked as 'proposed target' are design goals that will be evaluated experimentally. No measured results are presented at this stage.

KPI-001: Physical-to-Digital Synchronisation Latency

Definition	Elapsed time from a physical state change occurring on the AGV to the Digital Twin virtual model reflecting that cha
Unit	Milliseconds (ms)
Proposed Target	< 200 ms end-to-end under normal LAN conditions
Measurement Method	Timestamp physical event in ESP32 telemetry; record DT model update timestamp on RPi; compute difference.
Test Reference	EXP-001 (Research Experiment Plan)

KPI-002: Digital-to-Physical Command Latency

Definition	Elapsed time from operator issuing a command on the dashboard to the AGV physically executing the command.
Unit	Milliseconds (ms)
Proposed Target	< 500 ms end-to-end under normal LAN conditions
Measurement Method	Timestamp command issue in dashboard; record motor response timestamp from ESP32 acknowledgement.
Test Reference	EXP-001

KPI-003: MQTT Message Throughput

Definition	Number of MQTT messages successfully published and received per second on the local broker.
Unit	Messages per second (msg/s)
Proposed Target	> 50 msg/s sustained without message loss under normal load
Measurement Method	Monitor Mosquitto broker statistics; measure with MQTT load testing tool (e.g., mqtt-bench).
Test Reference	EXP-002

KPI-004: MQTT Packet Loss Rate

Definition	Percentage of MQTT messages transmitted that are not received by the subscriber under normal operating conditio
Unit	Percentage (%)
Proposed Target	< 0.1% at QoS 1; 0% at QoS 2 under normal LAN
Measurement Method	Assign sequence numbers to all messages; count gaps at subscriber.
Test Reference	EXP-002, EXP-003

KPI-005: Sensor Telemetry Update Rate

Definition	Rate at which the ESP32 transmits complete telemetry messages to the Raspberry Pi.
Unit	Hertz (Hz)
Proposed Target	≥ 10 Hz (configurable up to 20 Hz)
Measurement Method	Count UART messages received by RPi bridge per second; log sequence numbers.

Test Reference	TC-COMM-003

KPI-006: AGV Localisation Error (Odometry)

Definition	Euclidean distance between the AGV's estimated position (from odometry) and its ground-truth position at mission c
Unit	Centimetres (cm)
Proposed Target	< 10 cm per 2 m of travel (odometry without correction); < 5 cm with marker correction
Measurement Method	Measure physical AGV position with tape measure or grid markings; compare to estimated position logged by navig
Test Reference	EXP-005

KPI-007: Obstacle Avoidance Response Time

Definition	Elapsed time from an obstacle entering the sensor detection zone to the AGV motors halting.
Unit	Milliseconds (ms)
Proposed Target	< 150 ms (sensor polling + Safety Manager response + PWM update)
Measurement Method	Place obstacle in sensor range; timestamp detection event and motor-stop event from system logs.
Test Reference	EXP-006, TC-SAF-003

KPI-008: Mission Success Rate

Definition	Percentage of assigned missions that successfully reach COMPLETED state without requiring manual intervention.
Unit	Percentage (%)
Proposed Target	> 90% under normal operating conditions (no unexpected obstacles)
Measurement Method	Record mission outcomes in database; compute COMPLETED / (COMPLETED + FAILED) ratio over N trial runs.
Test Reference	EXP-006, TC-SYS-008

KPI-009: Raspberry Pi CPU Utilisation

Definition	Average CPU utilisation of the Raspberry Pi 5 across all cores during normal operation with all services running.
Unit	Percentage (%)
Proposed Target	< 60% average; < 90% peak during telemetry bursts
Measurement Method	Monitor with psutil or system health service; log to InfluxDB; analyse over 30-minute run.
Test Reference	EXP-007

KPI-010: Raspberry Pi Memory Utilisation

Definition	Percentage of available RAM used by all running services under normal operational load.
Unit	Percentage (%)
Proposed Target	< 70% of installed RAM
Measurement Method	Monitor with psutil; log to InfluxDB; check for memory leaks over 24-hour run.
Test Reference	EXP-007

KPI-011: AGV Battery Run-Time

Definition	Duration of continuous operation from full charge (100% SoC) to critical low battery (5% SoC) under representative
Unit	Minutes
Proposed Target	≥ 60 minutes (design target; actual value depends on battery capacity and load profile)
Measurement Method	Charge to full; run representative mixed-operation mission sequence; record total runtime until BATTERY_CRITICA
Test Reference	EXP-007

KPI-012: System Availability

Definition	Percentage of planned operational time during which all core services are running and the AGV is responsive.
Unit	Percentage (%)
Proposed Target	> 95% during planned testing sessions
Measurement Method	Track service uptime via health monitor; compute (total time – downtime) / total time.
Test Reference	TC-SYS-010

21. Functional Requirements

Embedded System (ESP32)

	Category	Requirement Statement
FR-001	Embedded	The ESP32 shall acquire wheel encoder pulse data from both drive wheels via hardware interrupts at the encode
	Embedded	The ESP32 shall compute real-time wheel RPM for each drive wheel from encoder pulse count and elapsed time
FR-003	Embedded	The ESP32 shall generate PWM signals for the left and right motor channels with a frequency between 1 kHz ar
	Embedded	The ESP32 shall read IMU accelerometer and gyroscope data over I2C at a minimum rate of 50 Hz and apply a
FR-005	Embedded	The ESP32 shall measure front obstacle distance using ultrasonic or ToF sensors and update the OBSTACLE_I
	Embedded	The ESP32 shall monitor battery voltage (and current if INA219 is used) at a minimum rate of 2 Hz.
FR-007	Embedded	The ESP32 shall implement independent PID closed-loop velocity controllers for the left and right wheels with co
	Embedded	The ESP32 shall serialise all sensor data, control state, and diagnostic information into a structured JSON mess
FR-009	Embedded	The ESP32 shall receive, parse, and validate command messages from the Raspberry Pi via UART and dispatc
	Embedded	The ESP32 shall operate the hardware watchdog timer; the main control loop shall feed the watchdog at a minim
FR-011	Embedded	The ESP32 shall store configurable parameters (PID tuning, obstacle thresholds, maximum velocity) in non-vola
	Embedded	The ESP32 shall include a diagnostics module that counts occurrences of each error event type and includes thi

AGV Navigation

	Category	Requirement Statement
FR-013	AGV	The system shall support Level 1 predefined waypoint navigation: the AGV shall follow a configurable sequence
	AGV	The Raspberry Pi navigation service shall compute an odometry-based position estimate (x, y, θ) from encoder t
FR-015	AGV	The navigation service shall generate wheel velocity setpoints to navigate the AGV from its current estimated po
	AGV	The navigation service shall detect mission completion when the estimated AGV position is within a configurable
FR-017	AGV	The system shall support three navigation modes: MANUAL, SEMI-AUTONOMOUS, and AUTONOMOUS MISS

Communication

	Category	Requirement Statement
FR-018	Comm	The Raspberry Pi 5 shall host an MQTT broker (Mosquitto) accessible on the local network, operating on port 18
	Comm	The UART-MQTT bridge service on the Raspberry Pi shall read telemetry from the UART serial port and publish
FR-020	Comm	The system shall publish heartbeat messages to the twinware/agv/{id}/heartbeat topic at 1 Hz.
	Comm	The system shall implement Last Will and Testament (LWT) on the MQTT connection such that the broker publis
FR-022	Comm	All MQTT clients shall implement automatic reconnection with exponential back-off (maximum interval: 60 secon

	Comm	Command messages shall be published to twinware/agv/{id}/command with QoS 2 to guarantee exactly-once de
FR-024	Comm	The Raspberry Pi UART bridge shall transmit heartbeat messages to the ESP32 over UART at a minimum rate

Edge Computing (Raspberry Pi)

	Category	Requirement Statement
FR-025	Edge	The Raspberry Pi shall run all core services as systemd managed units with automatic restart on failure.
	Edge	The telemetry processor shall parse incoming MQTT telemetry messages, validate required fields, and write to t
FR-027	Edge	The DT Sync Engine shall update the in-memory Digital Twin model with each new valid telemetry message and
	Edge	The health monitoring service shall collect Raspberry Pi CPU, memory, temperature, and disk metrics at 1 Hz ar
FR-029	Edge	The FastAPI backend shall provide REST API endpoints as defined in Section 13.2, accessible on the local netw
	Edge	The backend shall push real-time DT state updates to connected dashboard clients via WebSocket at a rate pro

Digital Twin

	Category	Requirement Statement
FR-031	DT	The Digital Twin shall maintain the AGV entity data model as defined in Section 11 and update all fields from inc
	DT	The Digital Twin shall track and transition between the operational states defined in Section 10.2 based on telem
FR-033	DT	The Digital Twin shall detect STALE state when no telemetry has been received for longer than a configurable th
	DT	The Digital Twin shall record all state transitions to the event store with timestamps.
FR-035	DT	The Digital Twin shall provide access to historical sensor data via the API backend for configurable time ranges.

Dashboard

	Category	Requirement Statement
FR-036	Dashboard	The dashboard shall display the real-time AGV position on a 2D warehouse map representation, updated within
	Dashboard	The dashboard shall display the current DT state (IDLE, MOVING, OBSTACLE_HALT, E-STOP, OFFLINE, etc.)
FR-038	Dashboard	The dashboard shall provide a mission creation interface where the operator can select source station, destinati
	Dashboard	The dashboard shall allow the operator to issue PAUSE, RESUME, and CANCEL commands for active missions
FR-040	Dashboard	The dashboard shall display time-series charts for at least the following metrics: speed, battery SoC, synchronis

Database

	Category	Requirement Statement
FR-041	DB	The system shall store all incoming telemetry records in InfluxDB with UTC timestamps and the AGV identifier a
	DB	The system shall store mission lifecycle records in SQLite with all state transition timestamps.

	DB	The system shall store all system events and alerts in SQLite with severity, type, source, and description.
FR-044	DB	The system shall implement configurable data retention policies: telemetry data shall be retained for at least 30 d

Safety

	Category	Requirement Statement
FR-045	Safety	The system shall implement all ten safety requirements defined in Section 17 (SAF-001 through SAF-010).
	Safety	Emergency stop state shall be clearly indicated on the dashboard with a prominent EMERGENCY STOP alert ba

Diagnostics

	Category	Requirement Statement
FR-047	Diag	The diagnostics panel shall display MQTT broker status, database status, backend service status, and Raspberr
	Diag	The system shall log all fault events, state transitions, command events, and reconnection events to rotating log

22. Non-Functional Requirements

	Category	Requirement Statement
NFR-001	Performance	The physical-to-digital synchronisation latency shall not exceed 200 ms under normal LAN operating conditi
	Performance	The digital-to-physical command latency shall not exceed 500 ms from dashboard submission to AGV exec
NFR-003	Performance	The ESP32 firmware shall transmit telemetry at a minimum rate of 10 Hz.
	Performance	The MQTT broker shall sustain a minimum message throughput of 50 messages per second without messa
NFR-005	Performance	The FastAPI backend shall respond to REST API GET requests within 200 ms under normal load (< 5 conc
	Reliability	The ESP32 shall recover from a transient UART communication fault (gap of < 5 seconds) automatically wit
NFR-007	Reliability	All Raspberry Pi services shall restart automatically within 30 seconds of an unexpected crash, managed by
	Reliability	The MQTT clients shall reconnect within 60 seconds of a network interruption using exponential back-off.
NFR-009	Reliability	The system shall maintain data integrity during planned and unplanned Raspberry Pi restarts (no database
	Reliability	The mission manager shall detect and handle mission timeout: if an EXECUTING mission does not complet
NFR-011	Scalability	The MQTT topic and API endpoint architecture shall be designed to support up to 10 AGVs without structura
	Scalability	The database schema shall not require structural migration to accommodate 6 months of telemetry data at 1
NFR-013	Maintainability	The ESP32 firmware shall be organised into clearly separated modules (as defined in Section 6) with no mo
	Maintainability	The Raspberry Pi software components shall be implemented in Python 3.10+ following PEP 8 style guideli
NFR-015	Maintainability	All configurable parameters (thresholds, rates, credentials, paths) shall be defined in configuration files, not
	Maintainability	The project shall maintain a structured Git repository with branch-per-feature workflow and descriptive comm
NFR-017	Usability	The dashboard shall be operable by a non-expert operator after a 10-minute orientation session.
	Usability	All alerts shall include a clear, human-readable description and recommended action. Generic error codes v
NFR-019	Usability	The dashboard shall be responsive and usable on a 13-inch or larger laptop screen at 1280×800 resolution
	Portability	The Raspberry Pi software stack shall be deployable via a documented setup script or Docker Compose co
NFR-021	Security	All MQTT clients shall authenticate with unique credentials. Shared credentials between services are not pe
	Security	Sensitive configuration values shall be managed via environment variables and shall not appear in Git histo
NFR-023	Testability	All synchronisation latency measurements shall be logged to the database with sufficient resolution (millisec
	Testability	The system shall provide a simulation mode where the ESP32 outputs pre-recorded telemetry replay to ena
NFR-025	Safety	The system shall default to a safe (motors stopped, alerts active) state in the absence of a valid configuratio

23. Use Cases

UC-001: Monitor AGV Real-Time State

Actor	Warehouse Operator / Researcher
Preconditions	System is powered on; all services running; AGV is connected and transmitting telemetry.
Main Flow	1. Operator opens dashboard in web browser. 2. Dashboard Overview screen loads with live warehouse map. 3. AGV position indicator updates as AGV moves (via WebSocket). 4. Battery level, speed, and state are shown in real time. 5. Operator views AGV Details screen for granular sensor data.
Alternative Flow	If AGV is OFFLINE, dashboard shows OFFLINE state badge and last-known position with stale indicator.
Exceptions	If dashboard cannot connect to backend, error banner displayed; auto-retry every 10 seconds.
Postconditions	Operator has current knowledge of AGV operational state.

UC-002: Create a Warehouse Mission

Actor	Warehouse Operator
Preconditions	Operator is authenticated. AGV is ONLINE_IDLE. No critical alerts active.
Main Flow	1. Operator navigates to Mission Management panel. 2. Selects source station from dropdown (e.g., Station A). 3. Selects destination station (e.g., Station D). 4. Sets priority (optional; defaults to 3). 5. Clicks Submit Mission. 6. Backend validates request; creates CREATED mission record. 7. Mission moves to QUEUED; dashboard shows queued mission.
Alternative Flow	If AGV is busy, mission is queued with QUEUED status.
Exceptions	If source = destination, form validation rejects submission. If AGV is in E-STOP or FAULT state, submission is rejected with error message.
Postconditions	Mission record created in database; mission queued for assignment.

UC-003: Execute Autonomous Mission

Actor	Mission Manager (automated, initiated by UC-002)
Preconditions	Mission in QUEUED state; AGV is ONLINE_IDLE.
Main Flow	1. Mission Manager assigns mission to AGV; state → ASSIGNED. 2. Navigation service computes route to source station. 3. Velocity commands sent to AGV; mission state → EXECUTING. 4. AGV travels to source station; arrival confirmed. 5. AGV waits configurable dwell time (cargo simulation). 6. AGV travels to destination station. 7. Arrival at destination confirmed; mission state → COMPLETED. 8. Event logged; dashboard notifies operator.

Alternative Flow	Obstacle detected: mission pauses → PAUSED; resumes when path clears. Battery critical during mission: mission → FAILED; return-to-charge mission auto-created.
Exceptions	Mission timeout exceeded: mission → FAILED; alert raised.
Postconditions	Mission record shows COMPLETED with timestamps; DT state returns to ONLINE_IDLE.

UC-004: Operator Stops AGV

Actor	Warehouse Operator
Preconditions	AGV is MOVING or MISSION_ACTIVE.
Main Flow	1. Operator clicks STOP button on dashboard. 2. Dashboard sends POST /api/v1/command with cmd=STOP. 3. Backend validates and publishes STOP to MQTT command topic (QoS 2). 4. UART bridge forwards STOP to ESP32 over UART. 5. ESP32 Command Parser receives STOP; Motor Controller sets PWM to 0. 6. ESP32 transmits acknowledgement via UART. 7. DT Sync Engine updates state to ONLINE_IDLE.
Alternative Flow	None
Exceptions	If MQTT publish fails, backend retries up to 3 times and returns error to dashboard.
Postconditions	AGV is stationary; active mission paused (not cancelled unless operator cancels).

UC-005: Detect Obstacle and Recover

Actor	System (automated)
Preconditions	AGV is MOVING on a mission.
Main Flow	1. Obstacle Sensor Manager detects distance < halt threshold. 2. Safety Manager sets EMERGENCY_STOP flag; motors halt. 3. OBSTACLE_HALT event published to MQTT alert topic. 4. DT state → OBSTACLE_HALT; dashboard shows obstacle alert. 5. Operator or automatic system waits for obstacle clearance. 6. Once path clear (distance > resume threshold), RESUME command issued. 7. Navigation service resumes mission; motors restart. 8. DT state → MISSION_ACTIVE; OBSTACLE_CLEARED event logged.
Alternative Flow	Operator manually cancels mission while in OBSTACLE_HALT.
Exceptions	If obstacle not cleared within mission timeout, mission → FAILED.
Postconditions	AGV resumes mission or mission is cancelled/failed; all events logged.

UC-006: Communication Failure and Recovery

Actor	System (automated)
Preconditions	AGV is in any operational state; UART or MQTT link is active.

Main Flow	1. UART heartbeat from RPi stops being received by ESP32. 2. ESP32 COMMS_TIMEOUT fires after 5 seconds; motors halt; COMMS_SAFE_STOP state entered. 3. Raspberry Pi UART bridge detects read timeout; logs UART_FAILURE event. 4. MQTT client publishes LWT: AGV OFFLINE. 5. DT state → OFFLINE; dashboard shows OFFLINE alert. 6. RPi attempts UART reconnection; ESP32 listens for heartbeat. 7. UART link restored; ESP32 clears COMMS_TIMEOUT; restores to previous operational state. 8. DT synchronisation resumes; ONLINE_IDLE or mission continues based on state.
Alternative Flow	Wi-Fi or network loss between dashboard and RPi: dashboard retries connection; AGV continues local operation.
Exceptions	None
Postconditions	Communication restored; all fault events logged; system operational.

UC-007: View Historical Telemetry and Analytics

Actor	Researcher / Project Evaluator
Preconditions	User authenticated; InfluxDB contains telemetry data.
Main Flow	1. User navigates to Analytics Dashboard. 2. Selects time range (last 30 minutes). 3. Selects metrics: speed, battery, latency. 4. Dashboard queries /api/v1/agv/{id}/telemetry with time and field params. 5. Backend queries InfluxDB; returns downsampled time-series data. 6. Dashboard renders line charts for each selected metric. 7. User exports selected data as CSV for further analysis.
Alternative Flow	None
Exceptions	If requested time range exceeds retention period, backend returns available data with a notice.
Postconditions	User has downloaded time-series data for offline analysis.

UC-008: Emergency Stop (Hardware)

Actor	Human (physical emergency stop button)
Preconditions	AGV is powered and in any operational state.
Main Flow	1. Operator or safety observer presses physical e-stop button. 2. Hardware interrupt fires on ESP32 GPIO within < 1 ms. 3. Safety Manager immediately sets PWM to 0 (motor stop). 4. ESP32 enters EMERGENCY_STOPPED state. 5. EMERGENCY_STOP event transmitted via UART in next telemetry cycle. 6. DT state → EMERGENCY_STOPPED; dashboard shows prominent E-STOP banner.
Alternative Flow	None
Exceptions	None
Postconditions	All motors stopped; system in EMERGENCY_STOPPED state until manual reset.

UC-009: View System Diagnostics

Actor	System Administrator / Maintenance Operator
Preconditions	Admin user authenticated; all services running.
Main Flow	1. Admin navigates to Diagnostics Panel. 2. Panel shows MQTT broker status, DB write rate, backend uptime, RPi CPU/memory/temperature. 3. Admin identifies service with high CPU utilisation. 4. Admin reviews log viewer for errors from that service. 5. Admin restarts service via dashboard (admin control).
Alternative Flow	None
Exceptions	If a critical service is not responding, restart control is greyed out; admin must SSH to RPi.
Postconditions	Service restarted; diagnostics normal.

UC-010: Low Battery Return to Charging

Actor	System (automated, triggered by Battery Monitor)
Preconditions	AGV is ONLINE_IDLE or MISSION_ACTIVE; battery SoC ≤ configurable low-battery threshold (default 20%).
Main Flow	1. Battery Monitor raises BATTERY_LOW alert; MQTT alert published. 2. Dashboard shows BATTERY_LOW warning badge. 3. If SoC falls to BATTERY_CRITICAL threshold (5%), Mission Manager auto-creates a RETURN_TO_CHARGE miss 4. If a mission is active, it is failed with reason BATTERY_CRITICAL. 5. Navigation service executes return-to-charge route to Charging Zone. 6. AGV arrives at Charging Zone; mission COMPLETED. 7. DT state → ONLINE_CHARGING; dashboard shows charging indicator.
Alternative Flow	If no return-to-charge route is available, motors stop; BATTERY_CRITICAL alert raised for manual intervention.
Exceptions	None
Postconditions	AGV at charging zone; system operator aware of charging status.

24. System State Machines

24.1 AGV Operational State Machine

State	Description	Entry Trigger	Exit Trigger(s)
INIT	Power-up self-test in progress	Power on	INIT_OK → IDLE; INIT_FAIL → FAULT
IDLE	All systems OK; no motion command	Mission complete / Stop command / Init OK	Mission assigned → ASSIGNED
ASSIGNED	Mission allocated; navigation computing route	Mission Manager assigns mission	Route ready → MOVING; Assign fail → IDLE
MOVING	AGV navigating toward current waypoint	Velocity command active	Waypoint reached → IDLE/MISSION_COMPLETE
OBSTACLE_HALT	Stopped due to obstacle in path	Obstacle < halt threshold while MOVING	Path clear + RESUME → MOVING; Timeout → FA
COMMS_SAFE_STOP	Stopped due to communication timeout	No UART heartbeat for > timeout	Comms restored + RESUME → IDLE
EMERGENCY_STOPPED	Hardware or software e-stop active	E-stop button pressed or EMERGENCY_STOP command	Manual reset + safety check → IDLE
CHARGING	At charging zone; battery being replenished	Arrived at Charging Zone + SoC < 80%	SoC ≥ 100% or departure command → IDLE
FAULT	Critical system fault; cannot operate	Critical sensor fail or init fail	Fault cleared + Reset → INIT

24.2 Mission State Machine

State	Entry Trigger	Valid Next States
CREATED	Operator submits mission via dashboard	QUEUED, CANCELLED
QUEUED	Mission accepted; AGV unavailable or lower priority	ASSIGNED, CANCELLED
ASSIGNED	AGV available; Mission Manager assigns	EXECUTING, CANCELLED
EXECUTING	Navigation service starts route execution	PAUSED, COMPLETED, FAILED, CANCELLED
PAUSED	Obstacle halt or operator pause	EXECUTING (resume), CANCELLED, FAILED
COMPLETED	AGV arrives at destination	— (terminal)
FAILED	Timeout, battery critical, unrecoverable fault	— (terminal)
CANCELLED	Operator cancels or system override	— (terminal)

24.3 Communication State Machine

State	Description	Entry Trigger	Exit Trigger
CONNECTED	UART + MQTT active; heartbeat exchanged	Initial connection established	Heartbeat miss > threshold → DEGRADED
DEGRADED	Intermittent message loss detected	1–2 heartbeat misses or elevated latency	Link restored → CONNECTED; Total loss → DI
DISCONNECTED	No communication; e.g., UART broken or MQTT offline	Heartbeat miss > timeout threshold	Reconnection successful → RECONNECTING
RECONNECTING	Active reconnection attempt in progress	DISCONNECTED state	Success → CONNECTED; Max retries → FAUL

24.4 Digital Twin Synchronisation State Machine

State	Description	Entry Trigger	Exit Trigger
SYNCHRONIZED	DT model fully current; latest telemetry applied	Telemetry received within staleness threshold	Telemetry age > staleness threshold → DELA
DELAYED	Telemetry arriving but with unusual latency	Individual telemetry latency > warning threshold	Latency returns to normal → SYNCHRONIZE
STALE	No telemetry received for > staleness threshold	Last telemetry age > staleness threshold (default 5 s)	New telemetry received → SYNCHRONIZED
OFFLINE	No heartbeat for extended period	Heartbeat absent > offline threshold (default 30 s)	Heartbeat resumes → SYNCHRONIZED
ERROR	DT Sync Engine encountered processing error	Parse error, DB write failure, etc.	Error cleared → SYNCHRONIZED

25. Sequence Diagram Descriptions

The following textual sequence descriptions define the interaction flows that shall be rendered as UML sequence diagrams in the detailed design documentation. Participants are abbreviated: **PHY** (Physical AGV), **ESP32** (Embedded Controller), **UART** (UART Link), **BRIDGE** (UART-MQTT Bridge on RPi), **MQTT** (Mosquitto Broker), **DT** (Digital Twin Engine), **DB** (Database), **API** (FastAPI Backend), **WS** (WebSocket), **DASH** (Dashboard).

SEQ-001 — AGV Startup Sequence

1. PHY powers on → ESP32 boots, FreeRTOS initialises.
2. ESP32 initialises all sensor modules (IMU, encoders, ultrasonic, battery monitor).
3. ESP32 sends INIT_STATUS message over UART: {msg_type: 'INIT', modules: [{'imu': 'OK'}, ...]}.
4. BRIDGE reads INIT_STATUS; publishes to MQTT twinware/agv/{id}/status.
5. DT receives status; sets state to ONLINE_IDLE; updates DB.
6. API broadcasts state update to WS clients.
7. DASH displays ONLINE_IDLE badge and ready indicator.
8. ESP32 begins transmitting TELEMETRY at 10 Hz.

SEQ-002 — Sensor Telemetry Flow

1. ESP32 assembles JSON telemetry from latest sensor readings every 100 ms.
2. ESP32 transmits via UART2 TX.
3. BRIDGE reads from UART serial buffer; adds ingestion_timestamp; publishes to MQTT twinware/agv/{id}/telemetry at QoS 1.
4. MQTT delivers to DT Sync Engine subscriber.
5. DT parses message; computes sync_latency_ms = ingestion_timestamp - msg_timestamp.
6. DT updates in-memory AGV entity with new sensor values.
7. DT writes telemetry record to InfluxDB.
8. DT writes sync latency to InfluxDB agv_latency measurement.
9. API pushes updated state to WS clients; DASH updates all panels.

SEQ-003 — Mission Assignment and Execution

1. DASH operator selects source/destination; clicks Submit.
2. DASH sends POST /api/v1/missions to API.
3. API validates parameters; creates CREATED mission record in SQLite.
4. Mission Manager is notified; checks AGV availability.
5. Mission Manager transitions mission to QUEUED → ASSIGNED.
6. Navigation service computes waypoint sequence.
7. Mission Manager publishes mission assignment to MQTT twinware/agv/{id}/mission.
8. Mission Manager sends first SET_VELOCITY command to MQTT command topic.
9. BRIDGE receives command; forwards via UART to ESP32.
10. ESP32 Command Parser decodes; PID controller activated; AGV starts moving.
11. ESP32 sends ACK in next TELEMETRY message.

12. DT state → MISSION_ACTIVE; DASH updates active mission card.

SEQ-004 — Obstacle Detection During Mission

1. AGV is MOVING; Obstacle Sensor Manager detects front distance < halt threshold.
2. Safety Manager immediately sets motor PWM to 0.
3. Safety Manager sets OBSTACLE_HALT flag.
4. ESP32 includes OBSTACLE_DETECTED=true and distance in next telemetry.
5. BRIDGE publishes telemetry; additionally publishes to MQTT twinware/agv/{id}/alert.
6. DT Sync Engine transitions state to OBSTACLE_HALT.
7. Mission Manager sets mission to PAUSED.
8. DASH displays OBSTACLE_HALT alert with sensor distance.
9. Obstacle is removed; next telemetry shows distance > resume threshold.
10. Mission Manager sends RESUME command via MQTT.
11. ESP32 resumes velocity setpoints; AGV moves.
12. DT state → MISSION_ACTIVE; mission → EXECUTING.

SEQ-005 — Emergency Stop (Software)

1. DASH operator clicks EMERGENCY STOP button.
2. DASH sends POST /api/v1/command with cmd=EMERGENCY_STOP.
3. API validates request; publishes to MQTT twinware/agv/{id}/command at QoS 2.
4. MQTT delivers to BRIDGE subscriber.
5. BRIDGE writes EMERGENCY_STOP command to UART.
6. ESP32 Command Parser receives; Safety Manager sets EMERGENCY_STOP immediately.
7. Motor Controller sets all PWM channels to 0.
8. ESP32 transmits ACK + state=EMERGENCY_STOPPED in telemetry.
9. DT transitions to EMERGENCY_STOPPED state.
10. Mission Manager marks active mission as FAILED with reason EMERGENCY_STOP.
11. DASH displays E-STOP banner; all mission controls disabled.

SEQ-006 — Communication Failure and Recovery

1. RPi reboots or UART bridge service crashes.
2. ESP32 UART receives no heartbeat for 5 seconds.
3. Safety Manager triggers COMMS_TIMEOUT safe state; motors halt.
4. MQTT LWT fires: twinware/agv/{id}/lwt published by broker.
5. DT Sync Engine receives LWT; transitions state to OFFLINE.
6. DASH shows OFFLINE badge and COMMUNICATION LOST alert.
7. BRIDGE service restarts (systemd auto-restart).
8. BRIDGE reconnects to MQTT; begins reading UART.
9. BRIDGE sends HEARTBEAT to ESP32 via UART.
10. ESP32 receives heartbeat; clears COMMS_TIMEOUT; transitions to IDLE.
11. DT receives telemetry; transitions to ONLINE_IDLE.
12. DASH clears OFFLINE badge; COMMUNICATION RESTORED event logged.

SEQ-007 — Digital Twin Synchronisation Latency Measurement

1. PHY state changes (e.g., velocity increases) at physical time T0.
2. ESP32 reads encoder at next 10 ms PID cycle; timestamp T1 (ESP32 uptime_ms).
3. ESP32 includes T1 in next telemetry message TX at time T2.
4. BRIDGE receives message at T3 (RPi system clock); notes ingestion latency = T3 - T2.
5. DT Sync Engine updates model at T4; records sync_latency = T4 - T1 (if clocks synchronised) or T4 - T3 + ingestion_latency.
6. WebSocket push to DASH at T5.
7. Dashboard renders updated position at T6.
8. End-to-end latency: T6 - T0 (requires synchronised clocks or physical measurement).
9. All latency values written to InfluxDB agv_latency measurement for analysis.

27. Error and Failure Handling

Failure Scenario	Detection Method	System Response	Recovery Procedure	Safe State
IMU sensor fault (I2C NAC	I2C read returns error; m	Flag degraded; IMU_FAULT f	Retry I2C after 500 ms; c	DEGRADED (IMU
Ultrasonic sensor malfunction	Out-of-range readings c	Disable affected sensor;	Sensor restored; re-enab	SENSOR_FAULT (rem
Motor failure (mechanical)	PID unable to achieve se	Raise MOTOR_FAULT; ha	Manual intervention; pow	EMERGENCY_STOPPED
Encoder failure	Encoder count static whil	PID output saturates; SP	Encoder restored; mission	EMERGENCY_STOPPED
Battery low	INA219 voltage < low thr	BATTERY_LOW alert; dash	Charge battery; request	Mission paused or cancelled
Battery critical	INA219 voltage < critical	BATTERY_CRITICAL alert;	Charge battery; system r	EMERGENCY_STOPPED
ESP32 firmware crash	Watchdog timeout fires (	ESP32 resets; sends INIT	Automatic via watchdog;	FAILED
UART communication failure	RPi bridge: read timeout	ESP32: heartbeat (COMMS_SAFE_STOP);	ESP32 reconnects; bridge	COMMS_SAFE_STOP
MQTT broker crash	Clients receive disconnect	All subscribers lose tele	System offline; Mosquitto	Recovers automatically
Wi-Fi network failure	Dashboard loses connecti	Dashboard shows "disco	MQTT reconnects to Wi-Fi;	Vehicle safe stop
Raspberry Pi power failure	All services stop; UART lin	ESP32: no CMD_VEL frame	Restart RPi; systemd brin	ESP32 safe stop (no
InfluxDB write failure	Write API returns error;	Log error; buffer data in	Restore InfluxDB service;	Data loss (temporary
Mission timeout	Mission in EXECUTING	Mission aborted; MISSION	Operator reviews log and	IDLE

28. Testing Strategy

Unit Testing — Firmware Modules

Test each ESP32 firmware module independently using simulated inputs (encoder pulses via GPIO stimulation, I2C mock data, UART loopback). Test each Python service on the Raspberry Pi independently using pytest and mock MQTT/DB clients.

Example test activities:

- PID controller step response test with simulated encoder feedback
- UART message serialiser/parser round-trip validation
- Battery SoC estimator lookup table validation
- Mission state machine transition unit tests
- DT Sync Engine state update unit tests

Integration Testing — Cross-Component

Test the integrated communication chain between ESP32 and Raspberry Pi: power on both devices; verify UART telemetry is received by the bridge; verify MQTT publication; verify DT model update.

Example test activities:

- ESP32 → UART → BRIDGE → MQTT → DT integration test
- Dashboard → REST API → MQTT → UART → ESP32 command flow test
- InfluxDB write + query integration test

Communication Testing — MQTT Protocol

Validate MQTT QoS behaviour, retained messages, LWT, and reconnection under simulated network faults.

Example test activities:

- QoS 0/1/2 message delivery verification
- LWT publication on unexpected disconnect
- Reconnection and message queue drain test

System Testing — End-to-End

Full system test with all hardware connected: execute complete mission workflow from creation through completion and verify all states, events, and database records.

Example test activities:

- Mission creation → execution → completion end-to-end
- Obstacle detection and recovery end-to-end
- Emergency stop and restart end-to-end
- Battery critical return-to-charge end-to-end

Digital Twin Testing — Synchronisation

Measure synchronisation latency between physical state changes and DT model updates across 100+ measurement samples. Verify state machine transitions are correctly reflected.

Example test activities:

- Latency measurement across 100 telemetry cycles
- State transition verification for all DT states
- Historical data query accuracy test

Performance Testing — KPIs

Measure all 12 KPIs defined in Section 20 under representative operational load. Record results in InfluxDB and generate analytical charts.

Example test activities:

- Sustained 10 Hz telemetry throughput for 30 minutes
- MQTT latency measurement under varying QoS
- RPi CPU/memory profile during full operation

Fault Injection Testing — Resilience

Deliberately induce each failure scenario in Section 27 and verify the system responds as specified (correct safe state, correct event logged, correct recovery).

Example test activities:

- UART cable disconnect and reconnect
- MQTT broker kill and auto-restart
- Simulated battery critical via parameter command
- Obstacle placement and removal during mission

User Acceptance Testing — Dashboard and Workflow

Have a user (team member or evaluator) unfamiliar with the system internals attempt standard operator tasks using only the dashboard and project documentation. Record task success rate and time.

Example test activities:

- Create and execute a mission without assistance
- Identify and acknowledge an alert
- Export telemetry data for a selected time range
- Perform emergency stop and system restart

29. Test Cases

The following test cases correspond to the functional and non-functional requirements. Actual Result and Status columns are to be completed during implementation. No results have been fabricated.

ID	Req ID	Description	Preconditions	Procedure	Expected Result	Actual Result	Status
TC-001	FR-001	Encoder pulse acquisition	AGV powered; motor running at known speed	Rotate wheel one revolution; count pulses reported by ESP32	1320 pulses ± 5% (PPR = 1320)	Pending	Pending
TC-002	FR-002	Wheel RPM computation	AGV powered; encoder functional	Command motor at 50% PWM; read RPM from ESP32	RPM within ±5% of expected RPM at that PWM	Pending	Pending
TC-003	FR-003	PWM generation on motor driver	ESP32 firmware loaded; motor connected	Command 25%, 50%, 75% duty cycle; measure output voltage	Output voltage proportional to PWM setting	Pending	Pending
TC-004	FR-004	IMU data acquisition	IMU connected; I2C functional	Place AGV stationary; read IMU for 10 s	Yaw, pitch, roll values stable; gyro Z near 0	Pending	Pending
TC-005	FR-005	Obstacle detection	Ultrasonic sensor connected; AGV moving	Place obstacle at 20 cm in front of sensor	Obstacle detected within 100 ms; AGV stops	Pending	Pending
TC-006	FR-006	Battery voltage monitoring	INA219 connected; battery fully charged	Read battery voltage	Reported voltage within ±0.1 V of measured voltage	Pending	Pending
TC-007	FR-007	PID velocity control	Motor + encoder functional; PID tuned	Command 60 RPM; observe for 30 seconds	Steady-state error < 5 RPM within 2 seconds; stead	Pending	Pending
TC-008	FR-008	UART telemetry transmission	ESP32 and RPi connected via UART	Run system for 10 seconds; count received messages	≥ 100 JSON messages received; no parse errors	Pending	Pending
TC-009	FR-009	UART command reception	UART link active	Send SET_VELOCITY command from RPi	AGV responds to new velocity command within 200 ms; AG	Pending	Pending
TC-010	FR-010	Watchdog timer	ESP32 running; watchdog enabled	Simulate main loop hang (debug pause > 2 s)	ESP32 resets automatically within 2 seconds	Pending	Pending
TC-011	FR-013	Level 1 waypoint navigation	AGV on warehouse floor; mission created	Create mission with 3 waypoints; start mission	AGV reaches all 3 waypoints in order; total travel	Pending	Pending
TC-012	FR-014	Odometry position estimation	AGV on flat surface; encoders calibrated	Move AGV 1 m straight; compare estimated position	Estimated position within 10 cm of actual 1 m	Pending	Pending
TC-013	FR-017	Navigation mode switching	System operational	Switch between MANUAL, SEMI-AUTONOMOUS, and AUTONOMOUS modes	Each mode activates correctly; invalid transitions	Pending	Pending
TC-014	FR-018	MQTT broker availability	Raspberry Pi running; Mosquitto installed	Connect MQTT test client from second LAN device	Connection accepted; broker version string received	Pending	Pending
TC-015	FR-019	UART-MQTT bridge	ESP32 transmitting; bridge service running	Monitor MQTT telemetry topic for 30 seconds	Bridge publishes ≥ 25 messages; payload matches UART	Pending	Pending
TC-016	FR-021	MQTT Last Will and Testament	Broker running; AGV connected	Forcibly kill AGV MQTT client; monitor broker	Broker publishes LWT message to offline topic within 60 seconds	Pending	Pending
TC-017	FR-022	MQTT reconnection	System operational with MQTT	Stop Mosquitto broker for 60 seconds, restart	All clients reconnect within 60 seconds; telemetry resu	Pending	Pending
TC-018	FR-023	QoS 2 command delivery	MQTT operational; simulated EMERGENCY_STOP	Publish command with QoS 2	Command received exactly once; no duplicate execu	Pending	Pending
TC-019	FR-025	Service auto-restart	systemd service files configured	Kill bridge process (SIGKILL); wait 30 seconds	Service restarted automatically within 30 seconds; no m	Pending	Pending
TC-020	FR-026	Telemetry database write	InfluxDB running; telemetry flowing for 5 minutes	Query InfluxDB for stored records	≥ 1500 records present; no gaps > 5	Pending	Pending
TC-021	FR-027	DT sync latency compensation	Full stack operational	Analyse sync_latency_ms values in InfluxDB	Mean latency < 200 ms; latency applied to each	Pending	Pending
TC-022	FR-031	DT AGV entity update	DT Sync Engine running; AGV moving	Change AGV speed; observe DT entity	DT entity velocity_x updates within 200 ms	Pending	Pending
TC-023	FR-032	DT state transitions	DT Sync Engine operational	Trigger each AGV state transition	DT state machine handles all defined transitions	Pending	Pending
TC-024	FR-036	Dashboard position display	Dashboard open; AGV moving	Move AGV 0.5 m; observe position on map	Position updates and moves 0.5 m on map	Pending	Pending
TC-025	FR-038	Mission creation via dashboard	Dashboard open; AGV idle	Create mission Station A → Station B via dashboard	Mission appears in queue within 2 seconds; AGV beg	Pending	Pending
TC-026	FR-040	Analytics chart rendering	InfluxDB contains 30 minutes of data	Open Analytics tab	Charts render within 3 seconds; data consistent with	Pending	Pending
TC-027	FR-041	InfluxDB telemetry persistence	InfluxDB running	Run system 10 minutes; restart RPi; query InfluxDB	Pre-restart data present after reboot; no data loss af	Pending	Pending
TC-028	FR-042	Mission record persistence	SQLite running	Complete 5 missions; restart RPi; query missions table	All 5 records present with correct state and tim	Pending	Pending
TC-SAF-001	SAF-001	Hardware emergency stop	AGV in motion	Press physical e-stop button	Motors stop within 50 ms; system enters EMERGENCY_STOPPED	Pending	Pending
TC-SAF-002	SAF-002	Software emergency stop	AGV in motion; dashboard connected	Click E-STOP on dashboard	Motors stop within 500 ms; E-STOP banner shown on	Pending	Pending

TC-SAF-003	SAF-003	Obstacle-triggered manual halt	AGV in motion; obstacle can be placed in front of ultrasonic sensor	Place hand/object at 10 cm from sensor, then remove	Motors stop within 150 ms; DT shows HALT state; recovery requires explicit command	TBD	TBD
TC-SAF-004	SAF-004	Communication timeout safe state	AGV in motion; UART bridge active	Stop UART bridge service; wait 6 seconds	AGV commands zero velocity; DT shows COMMS_SAFE_STOP state; DT sh	TBD	TBD
TC-SAF-005	SAF-005	Battery critical protection	System operational	Send SET_PARAM command that sets BATT_CRIT_V to 11.0 V (above actual	System halts motion; BATT_CRIT_V reverted to default after reboot	TBD	TBD
TC-SAF-006	SAF-006	Watchdog timer reset	ESP32 running with watchdog enabled	Inject artificial infinite loop in ESP32 debug UART command handler	ESP32 reboots within 3 seconds; DT logs reset event	TBD	TBD
TC-SAF-007	SAF-007	Maximum velocity enforcement	Motor controller active	Send SET_VELOCITY command with vx = 2.0 m/s (above configured	TX_Vx clamped to 0.5 m/s (max_velocity)	TBD	TBD
TC-PERF-001	KPI-001	P2D synchronisation latency	Full stack running; time sync established	Run 100 telemetry cycles; extract sync_latency_ms from InfluxDB	Mean < 200 ms; 95th percentile < 400 ms (results TB	TBD	TBD
TC-PERF-002	KPI-003	MQTT message throughput	Mosquitto running	Run mqtt-bench with 50 simultaneous publishers at QoS 1 for 60 s	Zero message loss; broker logs confirm thro	TBD	TBD
TC-PERF-003	KPI-009	RPi CPU utilisation	All services running; AGV in motion	Monitor with htop over 30 minutes during operation	Average CPU < 60%; peak < 90% (results TBD)	TBD	TBD
TC-SYS-001	FR-045	Full mission end-to-end	Complete hardware + software stack	Execute 10 missions; COMPLETED without manual intervention	10/10 missions complete; TBD% success rate	TBD	TBD
TC-SYS-002	NFR-006	UART fault recovery	UART operational	Disconnect UART cable for 3 seconds, then reconnect	System recovers within TBD; telemetry data resum	TBD	TBD

30. Development Roadmap

Phase 1: Research and Requirements | Duration: 1–2 weeks

Description	Literature review on AGV systems, digital twins, MQTT, edge computing. Define scope, write SRS.
Key Deliverables	• SRS document (this document) • Component selection shortlist • Risk register
Acceptance Criteria	SRS approved by academic supervisor.

Phase 2: Hardware Procurement | Duration: 1–2 weeks

Description	Order all mandatory components. Set up development workspace.
Key Deliverables	• All mandatory components received and inventoried • Development workbench set up
Acceptance Criteria	All mandatory BOM items procured.

Phase 3: Mechanical Assembly | Duration: 1–2 weeks

Description	Assemble AGV chassis. Mount motors, wheels, caster. Test motor rotation manually.
Key Deliverables	• Assembled AGV chassis • Motor rotation verified • Physical warehouse structure built
Acceptance Criteria	AGV rolls freely; both motors turn on manual power.

Phase 4: ESP32 Firmware — Core | Duration: 2–3 weeks

Description	Develop Motor Controller, Encoder Manager, PID Controller, IMU Manager, Obstacle Sensor Manager, Battery M
Key Deliverables	• Working motor control with PID speed regulation • Encoder-measured RPM logged to serial • IMU data streaming
Acceptance Criteria	AGV drives straight at commanded speed; PID stabilises within 2 seconds.

Phase 5: ESP32 Firmware — UART Comms | Duration: 1 week

Description	Develop UART Communication Manager, Command Parser, Safety Manager, Diagnostics Manager. Test UART
Key Deliverables	• JSON telemetry transmitted at 10 Hz to RPi serial port • Commands received and parsed
Acceptance Criteria	RPi receives and prints telemetry; AGV responds to velocity commands.

Phase 6: Raspberry Pi Edge Setup | Duration: 2 weeks

Description	Configure Raspberry Pi OS. Install Mosquitto, InfluxDB, SQLite, FastAPI, Python dependencies. Develop UART-M

Key Deliverables	• All services running as systemd units • Telemetry stored in InfluxDB • MQTT topics active
Acceptance Criteria	Telemetry visible in InfluxDB query client; MQTT topics verified with MQTT Explorer.

Phase 7: Navigation Service | Duration: 2 weeks

Description	Develop odometry position estimator, waypoint navigator, mission manager on RPi.
Key Deliverables	• AGV navigates Station A → Station B using Level 1 or Level 2 navigation
Acceptance Criteria	Mission executes end-to-end; arrival confirmed; position tracked.

Phase 8: Digital Twin Engine | Duration: 2 weeks

Description	Develop DT Sync Engine, AGV entity model, state machine, event store, WebSocket broadcast.
Key Deliverables	• DT reflects AGV state in real time • State transitions logged to DB
Acceptance Criteria	DT state matches physical AGV state; sync latency measured < 500 ms (initial target).

Phase 9: Backend and Database | Duration: 1–2 weeks

Description	Develop FastAPI REST endpoints, mission API, telemetry API, event API, auth.
Key Deliverables	• All API endpoints responding • Mission CRUD operations functional
Acceptance Criteria	All planned endpoints return expected responses in automated integration tests.

Phase 10: Dashboard | Duration: 2–3 weeks

Description	Build web dashboard: overview map, AGV details, mission panel, analytics charts, diagnostics, event log.
Key Deliverables	• Dashboard functional on LAN browser • All screens operational
Acceptance Criteria	UAT: non-team user can create and monitor a mission without assistance.

Phase 11: Integration and System Testing | Duration: 2 weeks

Description	Execute all test cases in Section 29. Perform fault injection tests. Measure all KPIs.
Key Deliverables	• Test case results recorded (Actual Result, Status) • KPI measurements logged
Acceptance Criteria	≥ 90% of test cases pass; all KPIs have measured values (even if targets not yet met).

Phase 12: Optimisation and Refinement | Duration: 1–2 weeks

Description	Address test failures. Tune PID. Optimise MQTT latency. Reduce CPU load. Improve dashboard UX.

Key Deliverables	• Updated test case results showing improved pass rate • Optimisation notes document
Acceptance Criteria	All mandatory FRs pass their test cases.

Phase 13: Documentation and Research Paper | Duration: 2 weeks

Description	Finalise all documentation. Prepare experimental results. Write research paper or project report.
Key Deliverables	• Final SRS (updated with actual results) • Research paper draft • Demo video
Acceptance Criteria	Documentation reviewed and approved by supervisor.

32. Risk Management

Risk ID	Risk Description	Prob	Impact	Severity	Mitigation Strategy	Contingency
RISK-001	Hardware component failure (motor, motor driver)	Medium	High	High	Procure 1 spare of each critical component; test all components before project start	Use simplified open-loop control; order replacement from local supplier
RISK-002	ESP32 GPIO damage due to wiring error	Medium	Medium	Medium	Use level shifters and current-limiting resistors; double-check wiring against schematic before power-on	Replace ESP32 board (low cost); verify wiring carefully
RISK-003	PID controller instability causing erratic AGV motion	High	Medium	High	Start with conservative Kp; increment gradually; use step response profile for tuning	Fall back to open-loop speed control; reduce maximum speed limit
RISK-004	Localisation accuracy insufficient for reliable station arrival	High	High	High	Implement Level 3 marker-based correction; test navigation accuracy before integration	Use Level 1 timed navigation for demonstration if Level 3 is not ready
RISK-005	MQTT latency exceeds target under WiFi load	Medium	Medium	Medium	Use Ethernet for RPi; keep broker local; use QoS 0 for high-frequency telemetry topics	Use QoS 1 for all topics; increase publish interval to reduce load
RISK-006	Raspberry Pi thermal throttling during simulation	Medium	Medium	Medium	Install active cooler; monitor temperature; reduce simulation complexity if needed	Reduce physics update rate; disable non-essential visualisation features
RISK-007	Battery run time shorter than expected	Medium	High	Medium	Test with actual load profile early; use higher capacity battery if needed	Tether AGV to external power supply for demonstration
RISK-008	Integration complexity causes schedule overrun	High	High	High	Prioritise MVP strictly; do not add optional features until MVP is complete and stable	Reduce scope to MVP; document remaining features as future work
RISK-009	Raspberry Pi OS or library compatibility issues	Low	Medium	Low	Use Raspberry Pi OS Bookworm (stable); pin all Python package versions in requirements.txt	Use Docker container with locked dependencies
RISK-010	Feature creep from optional components (LiDAR, SLAM, ROS 2)	High	High	High	Treat all optional features as separate phases; do not start until MVP is demonstrated	Remove optional features from project scope; focus on core deliverables

33. Cost Estimation

The following budget is an estimate for the Indian market as of 2026. Prices are approximate and subject to availability.

Category	Component	Estimated Cost (INR)	Type
Computing	Raspberry Pi 5 (4GB) + PSU + Cooler	■7,500	Mandatory
Computing	MicroSD Card (64GB A2)	■600	Mandatory
Embedded	ESP32 Development Board	■450	Mandatory
Drive System	DC Geared Motors with Encoders (x2)	■1,200	Mandatory
Drive System	Motor Driver (L298N/TB6612)	■200	Mandatory
Drive System	Wheels and Caster	■300	Mandatory
Sensors	IMU (MPU-6050)	■150	Mandatory
Sensors	Ultrasonic Sensors (x3)	■300	Mandatory
Sensors	Battery Monitor (INA219)	■250	Mandatory
Power	LiPo Battery (3S 2200mAh) + Charger	■1,800	Mandatory
Power	Buck Converter + Switches + Wiring	■400	Mandatory
Mechanical	Chassis + Fasteners + Warehouse Materials	■1,500	Mandatory
Subtotal	Minimum Viable Prototype	■14,650	
Advanced	Raspberry Pi Camera Module v3	■3,500	Optional
Advanced	2D LiDAR (RPLIDAR A1)	■9,500	Optional
Advanced	M.2 NVMe HAT + 128GB SSD	■2,500	Optional
Total	Advanced Prototype (All Options)	■30,150	

34. Minimum Viable Project (MVP)

To mitigate integration risk, the project must first achieve the MVP before attempting advanced features. The MVP constitutes the absolute baseline for a successful project submission.

- AGV moves under ESP32 control (motor PWM + encoder feedback).
- AGV detects obstacles and halts automatically (ultrasonic).
- ESP32 transmits telemetry and receives commands over UART.
- Raspberry Pi runs MQTT broker and telemetry bridge.
- Digital Twin Engine updates virtual state from telemetry.
- Dashboard displays real-time AGV state and warehouse map.
- Operator can execute a basic predefined mission (Level 1 navigation).
- System events and telemetry are logged to the database.
- Hardware emergency stop functions correctly.

36. Research Experiment Plan

EXP-001: Physical-to-Digital Synchronisation Latency Analysis

Objective	**Measure the end-to-end latency from physical state change to Digital Twin update.**
Procedure	Timestamp encoder events on ESP32. Timestamp DT model update on RPi. Measure offset over 1000 samples und
Target Metrics	Mean latency, median, 95th percentile, standard deviation (in ms).

EXP-002: MQTT QoS Level Comparison

Objective	**Evaluate the impact of QoS levels (0, 1, 2) on message delivery rate and latency.**
Procedure	Publish 1000 messages at each QoS level. Record delivery success rate and delivery time.
Target Metrics	Delivery rate (%), Average latency (ms) per QoS level.

EXP-005: Odometry Localisation Accuracy

Objective	**Quantify the accumulated positional error of the encoder+IMU odometry system.**
Procedure	Command AGV along a defined 5-metre square path. Measure final physical position. Compare to final DT estimate
Target Metrics	Euclidean error distance (cm), heading error (degrees).

EXP-007: Edge Node Resource Utilisation

Objective	**Analyse the computational load on the Raspberry Pi 5 while running the full Digital Twin stack.**
Procedure	Log CPU, memory, and disk I/O metrics during idle, single-AGV telemetry ingestion, and database query bursts.
Target Metrics	CPU utilisation (%), RAM usage (MB), I/O wait times.

39. Software Project Structure

```
TwinWare/ ■ ■■■ firmware/ # ESP32 C++ source code (PlatformIO/Arduino) ■ ■■■ src/ # Main application code ■ ■■■ lib/ # Custom libraries (Motor, Sensor, PID) ■ ■■■ platformio.ini # Build configuration ■ ■■■ edge/ # Raspberry Pi Python services ■ ■■■ bridge/ # UART-MQTT bridge ■ ■■■ dt_engine/ # Digital Twin synchronisation logic ■ ■■■ navigation/ # Odometry and path execution ■ ■■■ backend/ # FastAPI REST application ■ ■■■ api/ # Route definitions ■ ■■■ models/ # Database schemas (SQLAlchemy/Pydantic) ■ ■■■ services/ # Business logic ■ ■■■ dashboard/ # React/Vue web application ■ ■■■ src/ # UI components ■ ■■■ public/ # Static assets ■ ■■■ database/ # Init scripts for InfluxDB and SQLite ■■■ hardware/ # Schematics, BOM, 3D models ■■■ tests/ # Pytest suite and hardware integration tests ■■■ docker/ # Docker Compose files for deployment
```

46. References

[1] M. Grieves and J. Vickers, 'Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems,' in Transdisciplinary Perspectives on Complex Systems, Springer, 2017.

[2] IEEE Std 830-1998, 'IEEE Recommended Practice for Software Requirements Specifications', IEEE Computer Society, 1998.

[3] OASIS, 'MQTT Version 3.1.1 Plus Errata 01', OASIS Standard, 2015.

[4] W. Shi et al., 'Edge Computing: Vision and Challenges,' IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637-646, 2016.

[5] Espressif Systems, 'ESP32 Technical Reference Manual', 2023.

[6] Raspberry Pi Ltd, 'Raspberry Pi 5 Product Brief', 2023.

--- End of Document ---